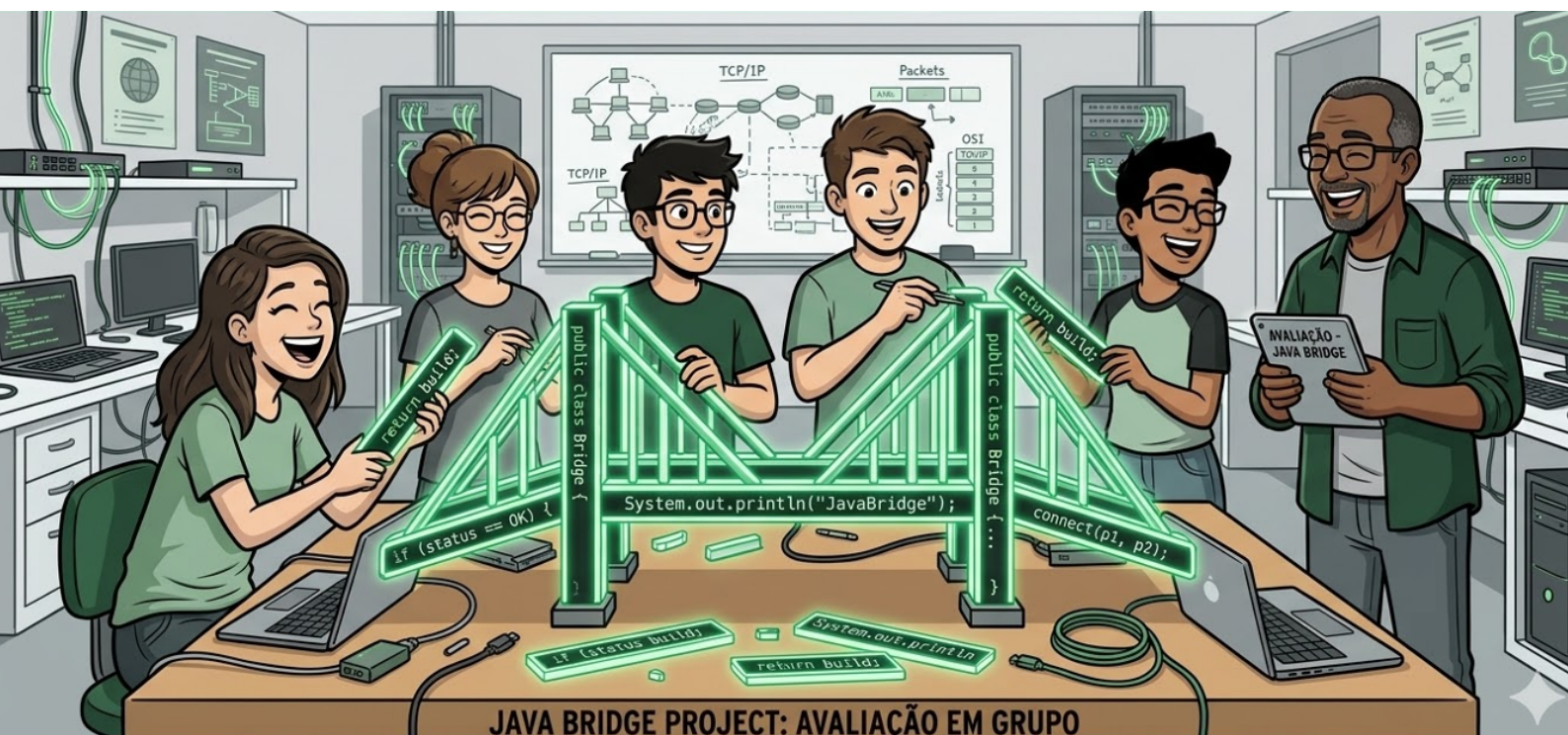


SERGIO SOUZA NOVAK, ADRIANO S. de O. BAILÃO

Tópicos em Confiabilidade e Qualidade de Software

1ª EDIÇÃO



Conteúdo

1	Introdução à Engenharia da Qualidade	6
1.1	Qualidade e Confiabilidade	7
1.1.1	Definições Básicas	7
1.1.2	Qualidade ao Longo do Tempo	8
1.2	Áreas de Atuação da Engenharia da Qualidade	8
1.2.1	Setores em que a Qualidade é Crítica	8
1.3	O Programa Apollo e a Engenharia de Confiabilidade	9
1.3.1	Contexto Histórico	9
1.3.2	Margaret Hamilton e o Pensamento de Confiabilidade	9
1.3.3	O Episódio no Simulador de Missão	9
1.3.4	O Problema Técnico Identificado	10
1.3.5	A Proposta de Solução de Margaret Hamilton	10
1.3.6	O Incidente Real na Apollo 8	11
1.3.7	Lições para a Engenharia Moderna e SRE	11
1.4	Modelo Tradicional: Sysadmin, Dev e Ops	12
1.4.1	Separação entre Dev e Ops	12
1.4.2	A Abordagem com Administradores de Sistemas	12
1.4.3	Conflitos Estruturais entre Dev e Ops	13
1.4.4	Respostas Típicas ao Conflito	14
1.5	A Abordagem do Google: Site Reliability Engineering (SRE)	14
1.5.1	Motivação para o Modelo SRE	15
1.5.2	Definição de SRE	15
1.5.3	Composição e Perfil das Equipes de SRE	15
1.5.4	Escalabilidade por Engenharia, não por Pessoas	16
1.5.5	A Regra dos 50%	16
1.5.6	Benefícios e Desafios do Modelo SRE	17
1.6	DevOps versus SRE	18
	Considerações Finais	19

2	Engenharia da Confiabilidade segundo Sommerville	20
2.1	Disponibilidade de Serviços e Desafios	20
2.2	Erro, Defeito e Falha	20
2.2.1	Erro Humano	21
2.2.2	Defeito de Sistema	21
2.2.3	Erro de Sistema e Falha	21
2.3	Nem Todo Defeito Gera Falha	21
2.4	Prevenção, Detecção e Tolerância a Defeitos	22
2.4.1	Prevenção de Defeitos	22
2.4.2	Detecção e Correção de Defeitos	22
2.4.3	Tolerância a Defeitos	22
2.5	Confiabilidade e Disponibilidade	23
2.5.1	Confiabilidade	23
2.5.2	Disponibilidade	23
2.6	Dependabilidade	23
2.7	Métricas de Confiabilidade e Disponibilidade	23
2.7.1	POFOD e ROCOF	24
2.7.2	MTTR e Disponibilidade	24
2.8	Visão Baseada em Entradas e Saídas	25
2.9	Considerações Finais sobre Sommerville	25
3	Arquiteturas Tolerantes a Defeitos e Diversidade de Software	26
3.1	Arquiteturas Tolerantes a Defeitos	26
3.1.1	Visão Geral em Blocos	26
3.2	Sistemas de Proteção	27
3.2.1	Diagrama de um Sistema de Proteção	28
3.3	Arquiteturas de Automonitoramento	28
3.3.1	Esquema de Dois Canais em Paralelo	29
3.4	Diversidade de Hardware e de Software	29
3.4.1	Diversidade de Hardware	29
3.4.2	Diversidade de Software	29
3.5	Programação N-versões	30
3.5.1	Diagrama Conceitual de N-versões	30
3.6	Conclusões sobre Diversidade e Tolerância a Defeitos	31
3.7	Hadoop: Tolerância a Defeitos em Escala Planetária	31
3.7.1	O Que é o Hadoop	31
3.7.2	Como Funciona o Cluster Hadoop	32
3.7.3	Tolerância a Defeitos no Hadoop	33

4	Monitoração Agregada: Princípios do Google SRE e Métricas de Confiabilidade	35
4.1	A Filosofia da Medição no Google SRE	35
4.2	Impactos das Falhas de Serviço	36
4.3	Downtime Não Planejado e Disponibilidade	36
4.3.1	Cálculo de Disponibilidade Baseada em Tempo	36
4.3.2	A Escala dos Noves	37
4.4	Confiabilidade em Sistemas Distribuídos	37
4.4.1	O Problema da Falha Localizada	38
4.5	Disponibilidade Baseada em Requisições	38
4.5.1	Exemplo de Cálculo de Orçamento de Erro	38
4.6	Monitoração de APIs REST e Protocolo HTTP	39
4.6.1	A Requisição HTTP	39
4.6.2	Métodos HTTP Principais	39
4.7	Classificação de Códigos de Status HTTP	40
4.8	Estudo de Caso: Caldeira Industrial Siderúrgica	40
4.8.1	Logs de Execução	40
4.8.2	Cálculo da Disponibilidade Observada	41
4.9	Service Level Objectives (SLO) e Error Budgets	41
4.10	Exercícios de Revisão	41
5	Métricas de Confiabilidade no Modelo Google SRE	43
5.1	A Filosofia da Medição e a Gestão de Riscos	43
5.1.1	Impactos das Falhas de Serviço	43
5.2	Downtime não Planejado como Métrica Central	44
5.2.1	Definição e Importância	44
5.3	Disponibilidade: A Matemática dos Noves	45
5.4	Limitações do Modelo de Disponibilidade Baseada em Tempo	46
5.5	APIs REST e Requisições: A Unidade Fundamental de Sucesso	46
5.5.1	O Protocolo HTTP e a Confiabilidade	46
5.6	Taxonomia SRE de Status Codes HTTP	47
5.7	Cálculo de Disponibilidade SRE: A Fórmula Fundamental	47
5.7.1	Exemplo de Grande Escala	48
5.8	SLO e Error Budget: O Equilíbrio entre Inovação e Estabilidade	48
5.9	Estudo de Caso Avançado: A Caldeira Industrial II	48
5.10	Exercícios de Aprofundamento	49

6	Estudo de Caso Prático: Análise de Logs de Servidor Web	51
6.1	Contexto e Origem dos Dados	51
6.1.1	Revisão: Distribuições Simétricas, Assimétricas e Boxplot	52
6.2	Processamento de Dados com Python e Pandas	55
6.3	Análise Estrutural e Descritiva	56
6.4	Análise de Latência: A Importância do Percentil 90	57
6.4.1	O Foco no Fim da Cauda	57
6.5	Calculando Disponibilidade (Availability) e Error Rate	57
6.5.1	O Teste do SLA e dos Noves	58
6.6	Discussão e Causa Raiz	58
7	Resposta a Emergências	60
7.1	Métricas Fundamentais: MTTF e MTTR	60
7.2	O Fator Humano: Seres Humanos Acrescentam Latência	61
7.2.1	O Manual de Regras (Runbook / Playbook)	61
7.3	Práticas de Preparação: A Cultura SRE	62
7.3.1	A Dinâmica do “Jogo”	62
7.4	As Coisas Quebram: A Vida É Assim	63
7.4.1	O Que Fazer Quando os Sistemas Falham	63
7.5	Estudos de Caso SRE: Taxonomia das Emergências	64
7.5.1	1. Emergência Induzida por Testes (O Caso do DB MySQL)	64
7.5.2	2. Emergência Induzida por Alterações de Configuração	65
7.5.3	3. Emergência Induzida por Processos Automáticos (Caso Diskerase)	66
7.6	Aprenda com o Passado: O Valor da Cultura <i>Blameless</i>	67
7.6.1	Testes Proativos e o Círculo Virtuoso da Resiliência Sistêmica	67
8	Engenharia de Segurança	69
8.1	O Software como Componente Crítico: O Caso de Varsóvia	69
8.2	O que é Segurança (<i>Safety</i>)?	70
8.2.1	<i>Safety</i> versus <i>Security</i>	71
8.3	A Segurança na Árvore da Dependabilidade	71
8.4	O Duplo Papel do Software na Segurança	71
8.4.1	Papel 1: Controle Direto	72
8.4.2	Papel 2: Monitoramento e Salvaguarda	72
8.5	A Falácia: Se é Confiável, é Seguro	72
8.6	Análise de Perigos: Antecipando o Desastre	73
8.6.1	Árvore de Falhas (<i>Fault Tree Analysis</i> — FTA)	74
8.6.2	FMEA: Análise de Modos de Falha e Efeitos	74
8.7	Categorias de Criticidade em Segurança	74

8.8	Requisitos de Segurança: O Poder do “Não Deve”	75
8.9	Arquitetura e Programação para Sistemas Seguros	76
8.9.1	Prevenção de Falhas (Fuga de Erros)	76
8.9.2	Tolerância a Falhas (Redundância e Votação)	76
8.9.3	O Estado Seguro (<i>Safe State</i>)	76
8.10	Software Crítico Primário vs. Secundário	77
8.11	Especificação Dirigida por Perigos	78
8.11.1	Terminologia Fundamental	78
8.11.2	O Fluxo da Especificação	78
8.12	Abordagens Práticas para Mitigação de Riscos	79
8.13	Cultura de Segurança e “Acidentes Normais”	79
8.14	O Registro de Perigos (<i>Hazard Registry</i>)	80
Referências Bibliográficas		81

Capítulo 1

Introdução à Engenharia da Qualidade

A Engenharia da Qualidade é uma área da engenharia dedicada ao planejamento, ao controle, à garantia e à melhoria da qualidade de produtos, serviços e processos. Enquanto a qualidade era, historicamente, tratada de forma predominantemente *inspetiva* — isto é, por meio de inspeções ao final do processo produtivo — a Engenharia da Qualidade traz uma abordagem proativa, baseada em métodos científicos, estatística e pensamento sistêmico.

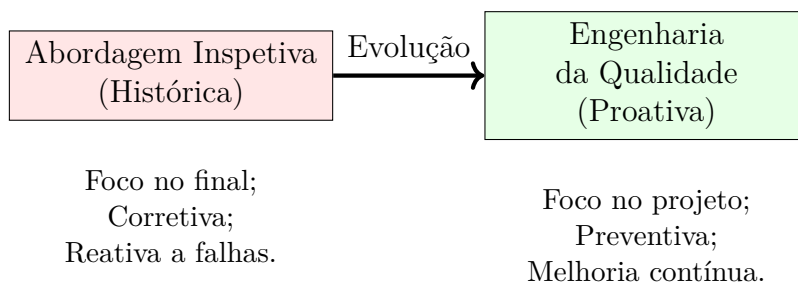


Figura 1.1: Evolução da gestão da qualidade: do modelo reativo ao proativo.

O ponto de partida é a compreensão de que **qualidade não é um acaso**, mas sim o resultado de processos bem projetados e bem controlados. Um produto ou serviço de qualidade é aquele que atende aos requisitos explícitos especificados em normas, contratos e regulamentos; que satisfaz às expectativas implícitas dos clientes, muitas vezes não completamente formalizadas; que apresenta desempenho consistente ao longo do tempo, com baixa variabilidade e taxa reduzida de falhas; e que é produzido de forma eficiente, com mínimo despídio de recursos.

Do ponto de vista da organização, a Engenharia da Qualidade também está fortemente ligada à competitividade. Empresas que dominam as ferramentas e conceitos de qualidade tendem a lançar produtos mais confiáveis e atraentes, reduzindo custos de re-

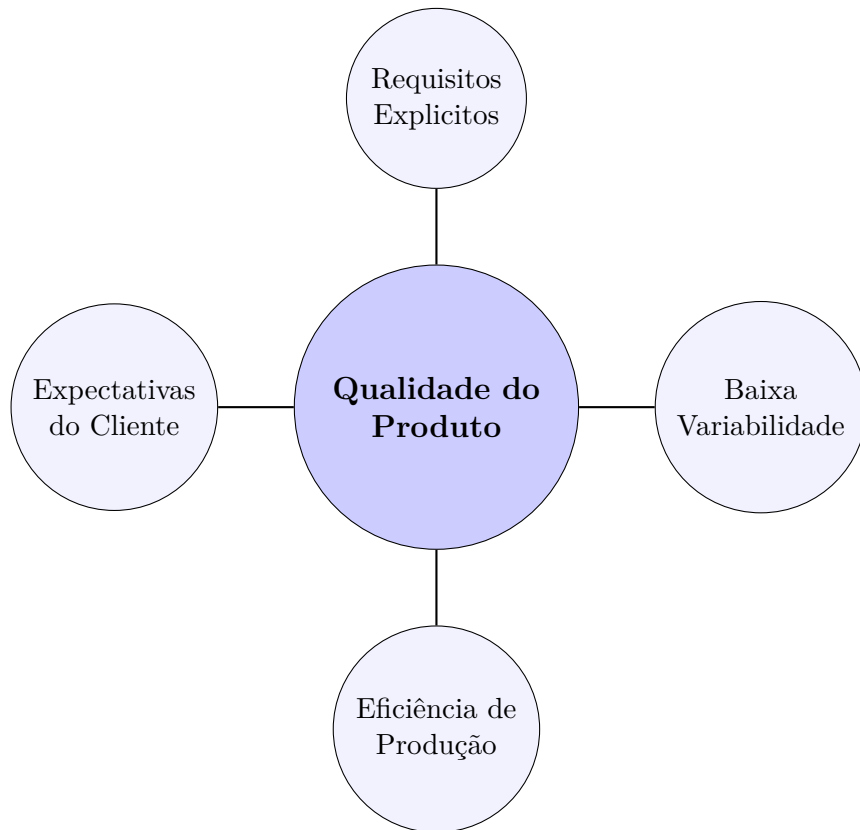


Figura 1.2: Os quatro pilares de um produto ou serviço de qualidade.

trabalho, garantia e suporte. Com isso, ganham reputação positiva junto aos clientes e se diferenciam em mercados cada vez mais exigentes.

1.1 Qualidade e Confiabilidade

1.1.1 Definições Básicas

Embora intimamente relacionadas, qualidade e confiabilidade não são sinônimos. De forma simplificada, podemos dizer que **qualidade** está associada ao grau em que um produto ou serviço atende às especificações e às expectativas do cliente *no momento em que é entregue*. Já a **confiabilidade** diz respeito à capacidade desse produto ou serviço de manter seu desempenho adequado *ao longo do tempo*, sob condições de uso específicas. Enquanto a qualidade pode ser observada num instante, a confiabilidade é necessariamente uma propriedade temporal.

Por exemplo, um sistema de software pode ser entregue com todas as funcionalidades combinadas (alta qualidade inicial), mas, se apresentar falhas frequentes em produção, terá baixa confiabilidade. Em contrapartida, um sistema levemente limitado em funcionalidades mas extremamente estável pode ser percebido como mais confiável e, em muitos

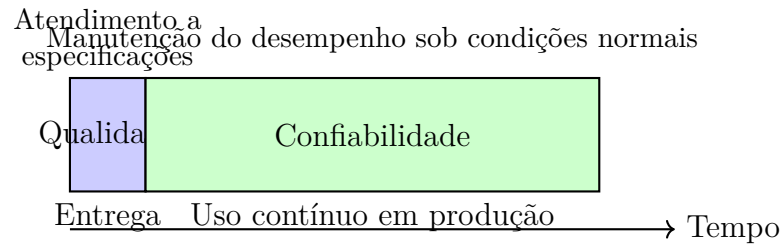


Figura 1.3: Qualidade (foco no momento da entrega) versus Confiabilidade (foco na duração).

contextos, mais valioso.

Ao longo desta disciplina, exploraremos justamente essa interseção entre qualidade de software, confiabilidade operacional e práticas modernas de engenharia, como o *Site Reliability Engineering* (SRE) adotado pelo Google ([BEYER; JONES et al., 2016](#); [GOOGLE SRE TEAM, 2024](#)).

1.1.2 Qualidade ao Longo do Tempo

1.2 Áreas de Atuação da Engenharia da Qualidade

1.2.1 Setores em que a Qualidade é Crítica

A Engenharia da Qualidade está presente em uma ampla variedade de setores econômicos. Tradicionalmente, associamos qualidade à indústria de manufatura, mas hoje ela é igualmente crítica nos **serviços** (bancos, hospitais, telecomunicações, educação, transporte), no **agronegócio** (controle de qualidade em insumos, produção, armazenagem e logística), na **Tecnologia da Informação** (desenvolvimento e operação de sistemas, aplicativos, plataformas em nuvem e serviços digitais) e no **setor público** (políticas de qualidade em serviços governamentais, transparência e foco no cidadão).

Em todos esses contextos, a Engenharia da Qualidade oferece ferramentas para mapear processos e identificar pontos críticos, medir desempenho por meio de indicadores, reduzir variabilidade e desperdícios, promover a melhoria contínua (*kaizen*) e apoiar a tomada de decisão baseada em dados.

No contexto específico da Engenharia de Software, a qualidade se manifesta, por exemplo, na definição de requisitos claros, na aplicação de boas práticas de desenvolvimento, na realização de testes sistemáticos e na operação confiável em produção.

1.3 O Programa Apollo e a Engenharia de Confiabilidade

1.3.1 Contexto Histórico

Um dos exemplos históricos mais interessantes na área de confiabilidade é o trabalho de **Margaret Hamilton** no programa Apollo, da NASA ([HAMILTON; ZELDIN, 1976](#); [FISHER, 2019](#)). Ela foi uma das engenheiras de software responsáveis pelo desenvolvimento dos sistemas de bordo utilizados nas missões que levaram o ser humano à Lua ([GARMAN, 1981](#)).

1.3.2 Margaret Hamilton e o Pensamento de Confiabilidade

Margaret Hamilton liderou a equipe de desenvolvimento de software no Laboratório de Instrumentação do MIT. Em uma época em que o termo “engenharia de software” ainda estava sendo definido, ela já defendia práticas rigorosas de projeto, verificação e tratamento de falhas — conceitos que hoje associamos diretamente à Engenharia de Confiabilidade e à SRE.

Seu trabalho enfatizava:

- a necessidade de **antecipar erros humanos** e falhas de operação;
- a importância de projetar sistemas que tolerem condições inesperadas;
- a criação de mecanismos de recuperação capazes de evitar que pequenos erros levem a desastres catastróficos.

1.3.3 O Episódio no Simulador de Missão

Um episódio marcante ocorreu durante testes no simulador do Apollo. Margaret levou sua filha Lauren ao trabalho, e, em determinado momento, a criança pressionou teclas do painel de controle (DSKY) de forma aleatória. Essa interação aparentemente inocente resultou na falha de uma missão simulada.

Do ponto de vista de engenharia, o episódio foi revelador:

- mostrou que era possível ativar, de maneira inadvertida, programas de navegação em momentos inadequados;
- evidenciou que erros simples de operação podiam ter consequências graves;
- indicou a necessidade de mecanismos adicionais de proteção.

Em vez de culpar o “usuário” (no caso, a criança), Hamilton interpretou o ocorrido como um **sintoma de fragilidade do sistema**. Essa postura é central em abordagens modernas de confiabilidade: em vez de supor que os operadores nunca erram, assume-se que erros humanos são inevitáveis e que o sistema deve ser projetado para lidar com isso.

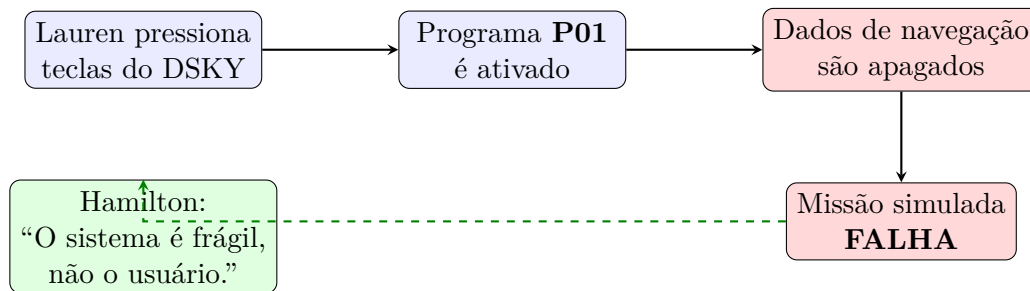


Figura 1.4: Fluxo do episódio no simulador: uma ação inocente desencadeou uma falha catastrófica, revelando fragilidades no design.

1.3.4 O Problema Técnico Identificado

A falha estava relacionada à seleção acidental do programa **P01**, que limpava os dados de navegação. Sem esses dados, o computador de bordo não conseguia pilotar a nave, o que, em uma missão real, poderia impedir o retorno à Terra.

Portanto, um simples comando, executado em um momento indevido, tinha potencial para comprometer toda a missão. Em termos de Engenharia de Confiabilidade, trata-se de um *ponto único de falha* (*single point of failure*) altamente sensível a erro humano.

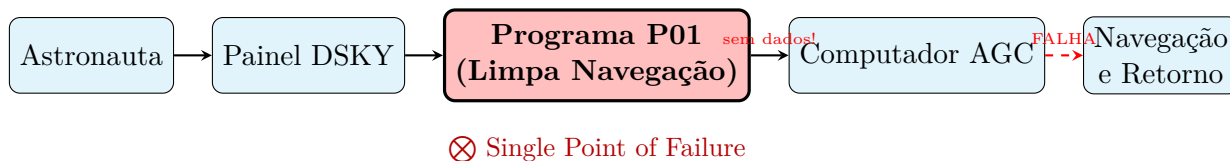


Figura 1.5: Ponto Único de Falha: a execução acidental do P01 era o elo mais fraco de toda a cadeia de navegação do Apollo.

1.3.5 A Proposta de Solução de Margaret Hamilton

Com mentalidade que hoje reconheceríamos como típica de SRE, Margaret propôs a inclusão de:

- um código de verificação de erro que detectasse o uso indevido do programa P01;
- bloqueios que impedissem a execução desse programa em fases críticas do voo;
- mecanismos de recuperação automática caso o comando fosse, ainda assim, acionado.

Sua proposta, entretanto, foi inicialmente rejeitada sob o argumento de que os astronautas eram altamente treinados e, portanto, não cometeriam esse tipo de engano. Essa confiança excessiva nos operadores contrasta com a visão contemporânea de SRE, que parte do princípio oposto: **erros humanos sempre acontecerão**.

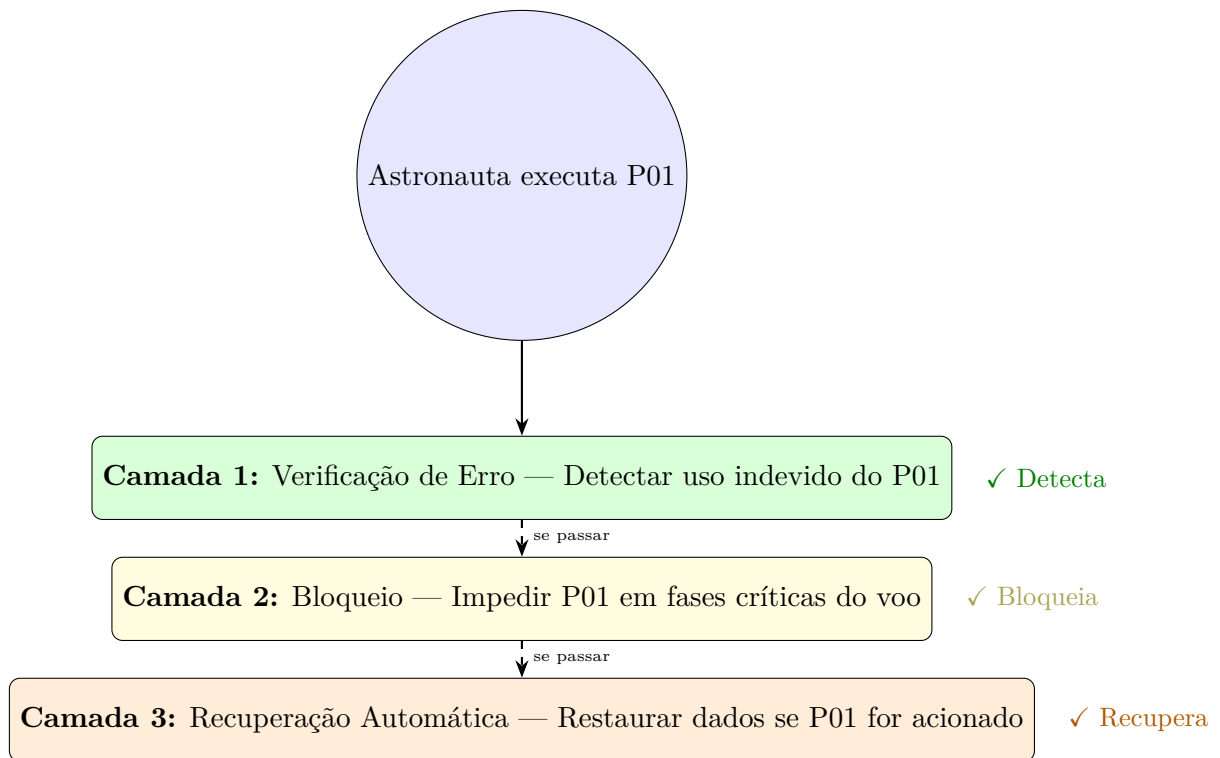


Figura 1.6: Camadas de defesa propostas por Margaret Hamilton: detecção, bloqueio e recuperação automática — o modelo de “Defesa em Profundidade”.

1.3.6 O Incidente Real na Apollo 8

Pouco tempo depois, durante a missão Apollo 8, o astronauta Jim Lovell acabou selecionando o programa P01 por engano, o que apagou os dados de navegação em plena missão. A situação foi crítica e exigiu um esforço significativo da equipe em solo para recuperar as informações e salvar a missão.

Esse incidente confirmou, de forma dramática, as preocupações de Margaret Hamilton. A documentação detalhada do sistema e o preparo da equipe permitiram que a missão fosse recuperada, mas o evento deixou claro que a abordagem de “confiar apenas na perfeição humana” é insustentável em sistemas de alta criticidade.

1.3.7 Lições para a Engenharia Moderna e SRE

O caso do programa Apollo ilustra princípios centrais que permeiam tanto a Engenharia de Confiabilidade quanto a SRE:

- **Erros humanos são inevitáveis:** sistemas devem ser projetados assumindo que pessoas vão se enganar, pressionar o botão errado ou esquecer um procedimento.
- **Sistemas críticos devem ser resilientes:** uma única ação incorreta não pode levar, sozinha, a um desastre.
- **Documentação e preparação são vitais:** conhecer bem o sistema e manter

registros atualizados aumenta as chances de recuperação.

- **Confiabilidade é requisito de projeto:** não se trata de um “extra” a ser considerado depois, mas de uma preocupação central desde a concepção do sistema.

Essas lições continuam extremamente atuais em um mundo em que sistemas computacionais controlam desde operações bancárias até infraestrutura crítica, como energia e telecomunicações.

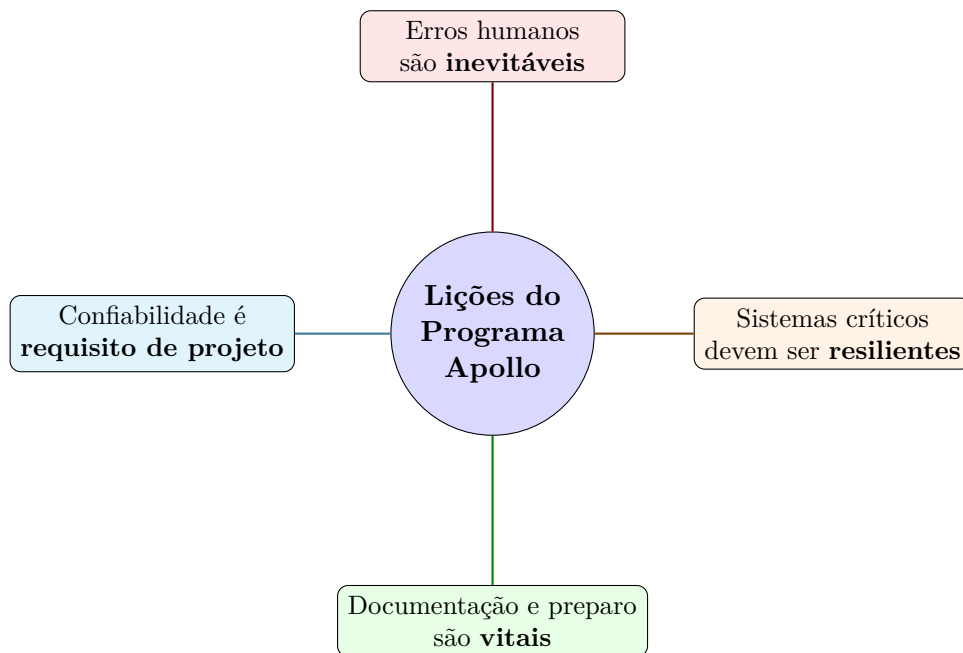


Figura 1.7: Mapa das lições fundamentais do Programa Apollo para a Engenharia de Confiabilidade moderna.

1.4 Modelo Tradicional: Sysadmin, Dev e Ops

1.4.1 Separação entre Dev e Ops

Historicamente, muitas organizações estruturaram suas equipes de tecnologia em dois grandes grupos: **Desenvolvimento (Dev)** e **Operações (Ops)**. Nesse modelo, os administradores de sistemas (*sysadmins*) eram responsáveis por manter serviços em funcionamento, enquanto os desenvolvedores criavam novas funcionalidades.

1.4.2 A Abordagem com Administradores de Sistemas

No modelo tradicional, os sysadmins:

- integram componentes de software;
- implantam serviços em produção;
- monitoram servidores e redes;

- respondem a incidentes e realizam manutenções.

À medida que o serviço cresce em número de usuários, complexidade e infraestrutura, costuma-se aumentar também o número de administradores de sistemas. Dessa forma, o custo operacional tende a crescer aproximadamente de forma linear com a escala do serviço.

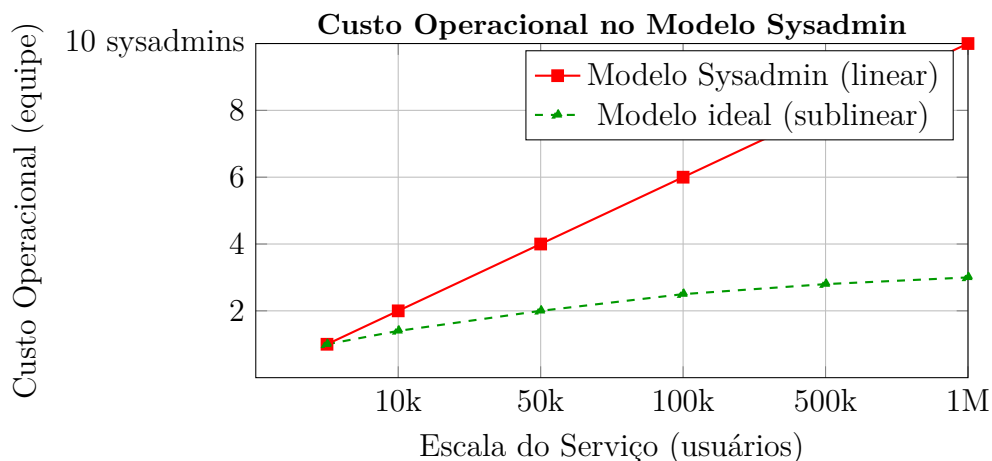


Figura 1.8: No modelo tradicional, o crescimento do serviço exige crescimento proporcional da equipe de operações, enquanto modelos baseados em automação (como SRE) escalam de forma sublinear.

Entre as vantagens desse modelo, podemos destacar:

- paradigma amplamente conhecido;
- facilidade de contratação de profissionais;
- existência de ferramentas consolidadas de administração de sistemas;
- separação clara de responsabilidades entre quem “constrói” e quem “opera”.

Por outro lado, o modelo gera conflitos importantes, especialmente em organizações que precisam inovar com rapidez.

1.4.3 Conflitos Estruturais entre Dev e Ops

A equipe de desenvolvimento, de maneira geral, é avaliada pela velocidade com que entrega novas funcionalidades e responde a demandas de negócio. Já a equipe de operações é cobrada pela estabilidade, disponibilidade e ausência de incidentes em produção.

Assim, emergem objetivos que podem ser conflitantes:

- **Dev** quer mudar o sistema com frequência para inovar;
- **Ops** prefere mudanças raras, cuidadosamente controladas, para preservar a estabilidade.

Como a maioria das falhas e interrupções está associada a mudanças em produção, a tendência natural de operações é criar barreiras, controles e processos de aprovação mais rígidos. Isso pode levar a:

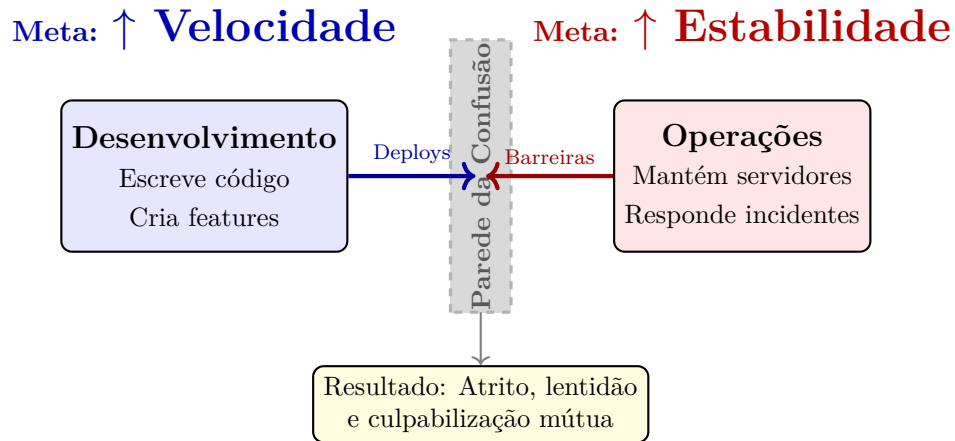


Figura 1.9: A “Parede da Confusão”: Dev empurra código para produção enquanto Ops ergue barreiras para proteger a estabilidade, gerando atrito.

- comunicação indireta e pouco transparente;
- ciclos de lançamento mais longos e menos frequentes;
- sensação de “guerra de trincheiras”, em que cada lado tenta defender seus interesses.

Na prática, o resultado é a redução da agilidade sem garantia de aumento real de confiabilidade.

1.4.4 Respostas Típicas ao Conflito

Para contornar os processos de aprovação longos, as equipes de desenvolvimento frequentemente:

- agrupam muitas mudanças em grandes *releases*, o que aumenta o risco de cada implantação;
- utilizam *feature flags* para ativar ou desativar recursos sem novo deploy;
- fragmentam o produto em múltiplos serviços menores, o que reduz o escopo de cada revisão, mas pode aumentar a complexidade geral do sistema.

Essas estratégias podem até aliviar alguns sintomas, mas não resolvem o problema estrutural de fundo: a falta de alinhamento entre os objetivos de desenvolvimento e de operações.

1.5 A Abordagem do Google: Site Reliability Engineering (SRE)

(BEYER; JONES et al., 2016; BEYER; MURPHY et al., 2018)

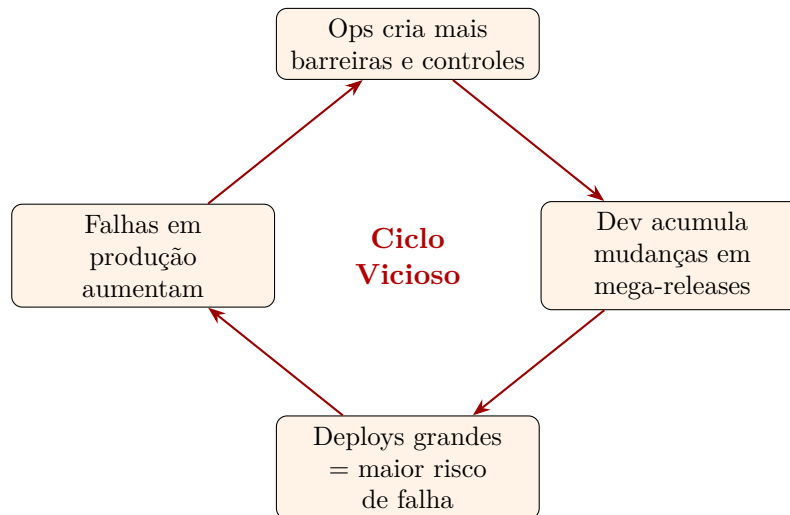


Figura 1.10: O ciclo vicioso do modelo tradicional: mais barreiras geram releases maiores, que geram mais falhas, que geram ainda mais barreiras.

1.5.1 Motivação para o Modelo SRE

O Google propõe uma resposta diferente a esse conflito histórico por meio do **Site Reliability Engineering (SRE)**. Em vez de manter uma divisão rígida entre Dev e Ops, o SRE busca tratar operações como um problema de engenharia de software.

1.5.2 Definição de SRE

Segundo a definição clássica do próprio Google:

SRE é o que acontece quando você pede a um engenheiro de software para projetar uma equipe de operações.

Isso significa que, em vez de depender de trabalho manual repetitivo e de processos puramente organizacionais, o SRE busca:

- automatizar o máximo possível das tarefas operacionais;
- usar métricas formais de confiabilidade (SLI, SLO e *error budget*) para orientar decisões;
- alinhar, por meio de código e processos, os objetivos de Dev e Ops.

1.5.3 Composição e Perfil das Equipes de SRE

As equipes de SRE do Google são majoritariamente formadas por engenheiros de software ou profissionais com formação muito próxima a essa. Tipicamente:

- 50–60% são engenheiros contratados pelo processo padrão de desenvolvimento;
- 40–50% possuem perfil técnico quase equivalente, com forte base em programação, sistemas Unix e redes.

Todos compartilham uma característica fundamental: **resolver problemas operacionais por meio de software**. Isso inclui:

- baixa tolerância a tarefas manuais repetitivas;
- foco em automação, observabilidade e escalabilidade;
- atuação próxima às equipes de produto, reduzindo barreiras organizacionais.

Composição Típica de uma Equipe SRE



Unix, Programação, Sistemas, Redes, Observabilidade, Automação

Figura 1.11: A equipe SRE combina engenheiros de software com profissionais de sistemas de perfil equivalente. O traço comum é a mentalidade de resolver problemas operacionais via código.

1.5.4 Escalabilidade por Engenharia, não por Pessoas

Um princípio central da abordagem SRE é que serviços de sucesso não devem exigir crescimento proporcional da equipe de operações. Em modelos tradicionais, mais usuários quase sempre significam mais servidores, mais complexidade e, conseqüentemente, mais pessoas para administrar tudo isso.

No modelo SRE, busca-se o oposto:

- investir continuamente em automação;
- reduzir o trabalho manual por meio de ferramentas;
- projetar serviços que se auto-recuperem e se auto-ajustem sempre que possível.

Graficamente, a ideia é que a curva de carga operacional ao longo do tempo seja decrescente ou, pelo menos, não cresça na mesma proporção que o tráfego.

1.5.5 A Regra dos 50%

Para garantir que o foco em engenharia não seja engolido pelas demandas do dia a dia, o Google adota uma regra explícita:

Nenhuma equipe de SRE deve gastar mais de 50% do tempo em atividades operacionais.

Em outras palavras, pelo menos metade do tempo da equipe deve ser dedicada a:

- desenvolver automações;
- melhorar a arquitetura dos serviços;
- eliminar causas-raiz de incidentes recorrentes;

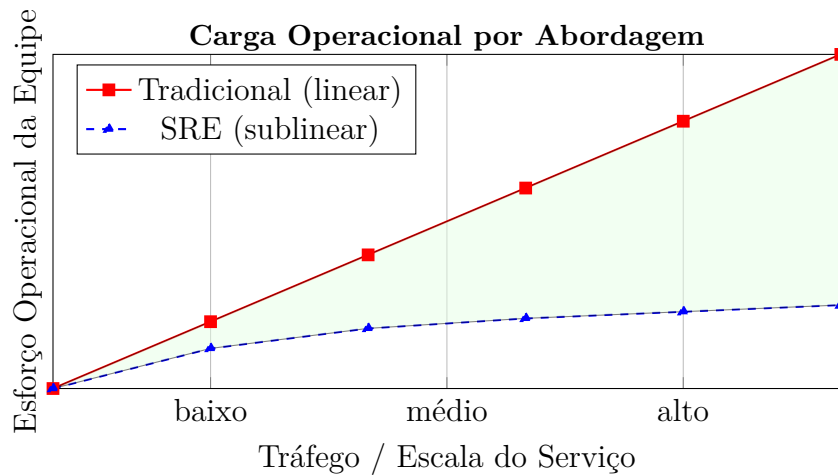


Figura 1.12: Enquanto o modelo tradicional exige crescimento de equipe proporcional ao tráfego, o modelo SRE investe em automação para manter o esforço operacional sublinear. A área verde representa a economia de esforço humano.

- aprimorar monitoração e ferramentas internas.

Quando esse limite é excedido por muito tempo, parte das tarefas operacionais é devolvida às equipes de desenvolvimento, forçando um redesenho do sistema para reduzir a necessidade de intervenção manual.

A Regra dos 50% de Tempo SRE

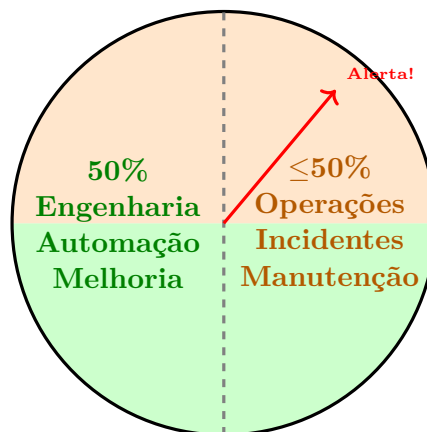


Figura 1.13: O tempo da equipe SRE deve ser dividido igualmente entre engenharia/automação e atividades operacionais. Se as operações consumirem mais de 50%, o sistema precisa ser redesenhado.

1.5.6 Benefícios e Desafios do Modelo SRE

Entre os principais benefícios do modelo SRE, destacam-se:

- menor custo operacional em larga escala;

- maior aceitação de mudanças frequentes, sem sacrificar a confiabilidade;
- compartilhamento de conhecimento entre SRE e desenvolvimento;
- cultura de aprendizado contínuo a partir de incidentes.

Por outro lado, há desafios significativos:

- dificuldade em contratar profissionais com perfil adequado;
- necessidade de apoio forte da alta gestão para sustentar decisões impopulares (como congelar lançamentos quando o *error budget* é consumido);
- mudança cultural profunda em organizações acostumadas ao modelo tradicional de Dev e Ops separados.

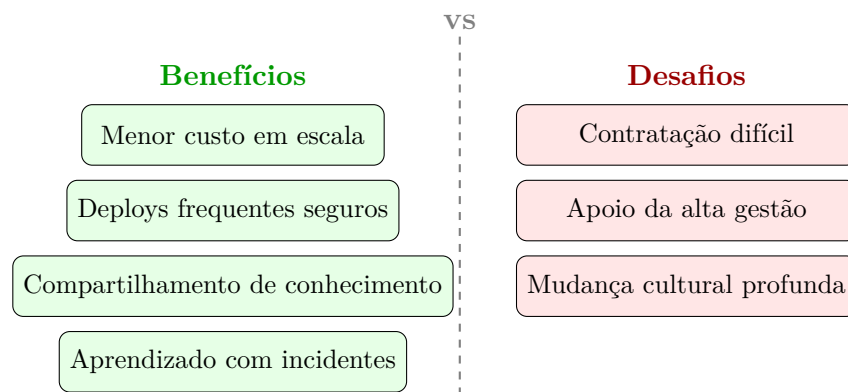


Figura 1.14: Benefícios e desafios do modelo SRE. A adoção exige mudança cultural além de mudança técnica.

1.6 DevOps versus SRE

Por fim, é importante diferenciar, ainda que brevemente, os conceitos de **DevOps** (KIM et al., 2018; HUMBLE; FARLEY, 2010) e **Site Reliability Engineering** (BEYER; JONES et al., 2016).

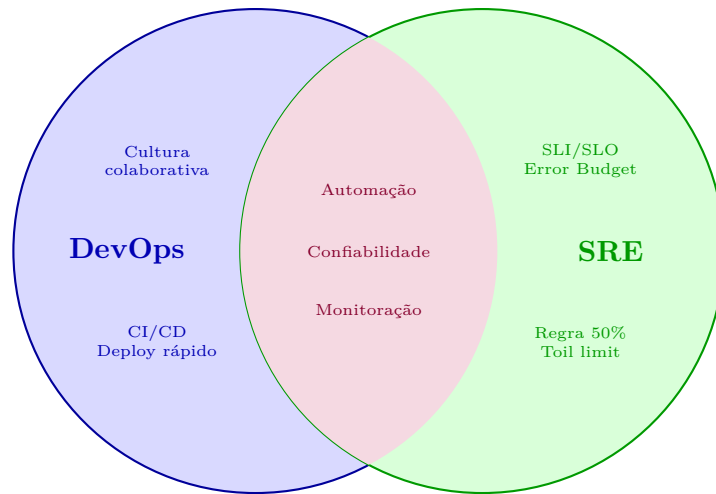
DevOps pode ser entendido como uma **cultura** e um **conjunto de práticas** que incentivam a colaboração entre desenvolvimento e operações, com foco em:

- automação de *deploy* e integração contínua;
- redução do tempo entre a ideia e sua disponibilização em produção;
- quebra de silos organizacionais.

Já o SRE, especialmente na formulação do Google, é uma **disciplina de engenharia** com práticas mais prescritivas, incluindo:

- uso sistemático de SLI, SLO e *error budget*;
- limites explícitos para tempo gasto com operações manuais;
- organização de equipes e responsabilidades bem definidas.

Podemos dizer que o SRE é uma *implementação concreta* de muitos dos princípios DevOps, com ênfase particular na confiabilidade de serviços em larga escala.



SRE = implementação concreta de princípios DevOps

Figura 1.15: Diagrama de Venn ilustrando a relação entre DevOps e SRE. DevOps define a cultura e práticas gerais; o SRE do Google representa uma implementação prescritiva dessas ideias com foco em confiabilidade.

Considerações Finais

Este texto expandiu o conteúdo apresentado nos slides da aula de introdução à disciplina de Engenharia da Qualidade e Confiabilidade. A partir de exemplos históricos, como o programa Apollo, e de práticas contemporâneas, como o SRE do Google, buscamos mostrar que confiabilidade não é apenas um tema técnico, mas também organizacional e cultural.

Nas próximas aulas, aprofundaremos esses conceitos, explorando métricas quantitativas, arquiteturas tolerantes a falhas, diretrizes de projeto e ferramentas práticas que permitem projetar, construir e operar sistemas de software com alta qualidade e confiabilidade.

Capítulo 2

Engenharia da Confiabilidade segundo Sommerville

([SOMMERVILLE, 2019](#))

A seguir, apresentamos um texto de apoio baseado nos slides da aula *Engenharia da Confiabilidade Sommerville*, aprofundando conceitos como erro, defeito, falha, confiabilidade, disponibilidade, dependabilidade e métricas associadas.

2.1 Disponibilidade de Serviços e Desafios

A sociedade moderna depende intensamente de sistemas de software para atividades profissionais e pessoais. Espera-se que serviços digitais estejam disponíveis de forma praticamente contínua: no início da manhã, à noite, em finais de semana e feriados. Além disso, esses sistemas devem preservar dados pessoais e corporativos e executar suas funções sem falhas aparentes.

Apesar dos avanços em linguagens de programação, processos de garantia da qualidade e técnicas de engenharia de software, falhas ainda ocorrem, a disponibilidade pode ser comprometida e resultados incorretos podem ser produzidos. A engenharia da confiabilidade busca reduzir a frequência e o impacto desses eventos.

2.2 Erro, Defeito e Falha

Sommerville distingue claramente entre três conceitos fundamentais: *erro humano* (*engano*), *defeito de sistema* (*bug*) e *falha de sistema*.

2.2.1 Erro Humano

Um erro humano é uma ação ou decisão equivocada que introduz um defeito em um sistema. Por exemplo, ao implementar a lógica de uma estação meteorológica, um desenvolvedor pode definir que o horário de transmissão é sempre o horário atual mais uma hora, sem considerar a transição de 23h para 0h. O equívoco acontece no nível cognitivo, antes mesmo da execução do programa.

2.2.2 Defeito de Sistema

O defeito (bug) é a materialização desse erro no código ou na estrutura do sistema. Ele representa uma característica física ou lógica que, quando executada, pode levar o sistema a um estado incorreto. Em termos de código, isso pode se manifestar como ausência de uma verificação de limites, uso de variáveis não inicializadas, acesso indevido a memória, entre outros.

2.2.3 Erro de Sistema e Falha

Quando um defeito é exercitado durante a execução, o sistema pode entrar em um *estado de erro*, isto é, um estado interno incorreto. Esse erro nem sempre resulta imediatamente em falha observável, mas, caso se propague até a interface com o usuário, tem-se uma *falha*: o serviço prestado não corresponde ao comportamento esperado.

Em um sistema de agendamento, por exemplo, um horário interno inválido (24:30 em um relógio de 24 horas) representa um erro de sistema. Se o escalonador não conseguir transmitir dados por causa desse valor inválido, ocorre uma falha, com perda de informações e interrupção do serviço.

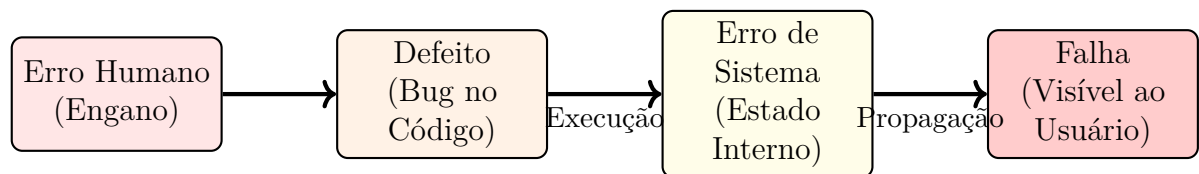


Figura 2.1: A cadeia Erro-Defeito-Falha clássica da engenharia de confiabilidade.

2.3 Nem Todo Defeito Gera Falha

É importante notar que defeitos não resultam necessariamente em erros, e erros não resultam necessariamente em falhas. Um trecho defeituoso pode jamais ser executado em um determinado perfil de uso; um erro de estado pode ser sobrescrito ou corrigido antes

de afetar a saída; mecanismos de tolerância a falhas podem detectar e recuperar o sistema antes que o usuário perceba qualquer problema.

Daí surgem três estratégias complementares para aumentar a confiabilidade:

- prevenir defeitos, por meio de boas práticas de projeto e implementação;
- detectar e corrigir defeitos antes da entrega, com verificação e validação sistemáticas;
- tolerar defeitos remanescentes, projetando sistemas capazes de se recuperar de estados indesejados.

2.4 Prevenção, Detecção e Tolerância a Defeitos

2.4.1 Prevenção de Defeitos

Na prevenção, procura-se evitar que erros humanos se transformem em defeitos no código. Isso inclui o uso de linguagens fortemente tipadas, padrões de projeto que reduzem complexidade e a adoção de construtos menos propensos a erros (por exemplo, evitando manipulação direta de memória quando não estritamente necessária).

2.4.2 Detecção e Correção de Defeitos

Mesmo com prevenção, defeitos permanecem. A detecção envolve atividades de verificação e validação (V&V), como análise estática, revisões de código e testes sistemáticos. O objetivo é encontrar e remover o máximo possível de defeitos antes que o sistema entre em operação, aumentando a confiabilidade percebida pelos usuários.

2.4.3 Tolerância a Defeitos

Em sistemas que exigem níveis muito altos de disponibilidade, assume-se que alguns defeitos sempre existirão. A solução é incorporar mecanismos de detecção de estados inválidos, tratamento de exceções, redundância e recuperação automática, de modo que o serviço continue sendo prestado mesmo na presença de falhas internas.

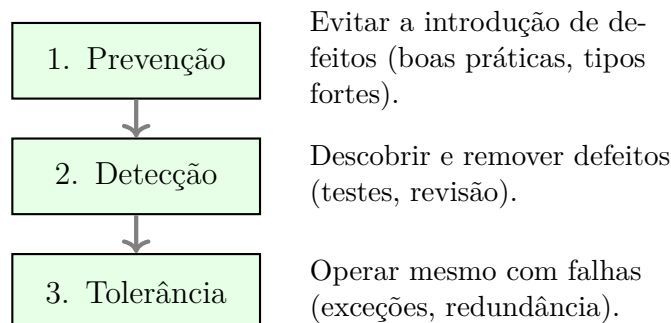


Figura 2.2: Estratégias complementares para lidar com defeitos ao longo do ciclo de vida.

2.5 Confiabilidade e Disponibilidade

Confiabilidade e disponibilidade são conceitos relacionados, mas distintos.

2.5.1 Confiabilidade

Confiabilidade é a probabilidade de operação *isenta de falhas* ao longo de um período de tempo especificado, em determinado ambiente, para um propósito específico. Ela está associada sobretudo à continuidade do serviço: por quanto tempo o sistema consegue operar sem apresentar problemas.

2.5.2 Disponibilidade

Disponibilidade é a probabilidade de que o sistema esteja operacional no momento em que for demandado. Um sistema pode sofrer falhas relativamente frequentes, mas recuperar-se rapidamente, apresentando alta disponibilidade ao longo do tempo. Em muitos contextos, usuários preferem uma falha rápida e segura, com rápida recuperação, a uma falha rara porém catastrófica.

Além disso, o impacto de uma falha depende fortemente do momento em que ela ocorre. Uma hora de indisponibilidade durante a madrugada pode causar menos perturbação do que alguns poucos minutos de queda em horário comercial.

2.6 Dependabilidade

Sommerville utiliza o termo *dependabilidade* para descrever a propriedade de um sistema na qual se pode confiar justificadamente. Ela engloba:

- disponibilidade;
- confiabilidade;
- segurança (*safety*), relacionada à ausência de consequências catastróficas;
- proteção (*security*), ligada à resistência a ataques e intrusões.

A dependabilidade é uma propriedade holística: depende não apenas do software, mas também do hardware, dos operadores e dos processos organizacionais. Requisitos funcionais e não funcionais precisam refletir metas claras de confiabilidade e disponibilidade, bem como mecanismos de verificação, recuperação e proteção.

2.7 Métricas de Confiabilidade e Disponibilidade

Para quantificar confiabilidade e disponibilidade, utilizam-se métricas como POFOD, RO-COF, MTTF, MTTR e AVAIL.

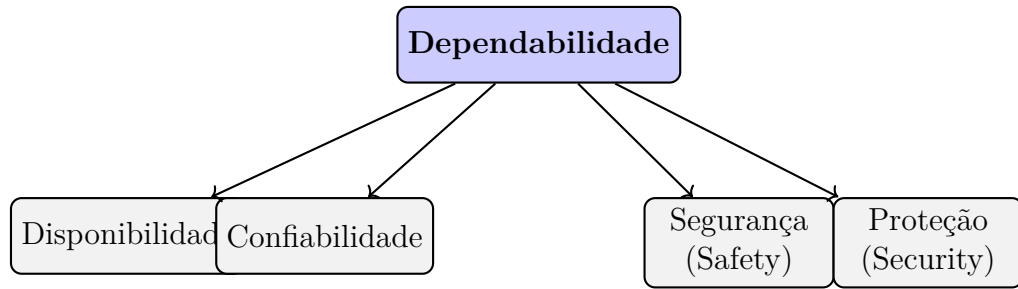


Figura 2.3: Os principais atributos que compõem a Dependabilidade de um sistema.

2.7.1 POFOD e ROCOF

A *Probabilidade de Falha sob Demanda* (POFOD) mede a probabilidade de uma requisição de serviço resultar em falha. É especialmente útil em sistemas de proteção acionados esporadicamente, como freios de emergência ou sistemas de backup.

Já a *Taxa de Ocorrência de Falhas* (ROCOF) indica o número esperado de falhas em um determinado período de tempo. Seu inverso é o *Tempo Médio entre Falhas* (MTTF). Em sistemas de uso contínuo, como servidores, ROCOF e MTTF fornecem uma visão clara da frequência com que problemas ocorrem.

2.7.2 MTTR e Disponibilidade

O *Tempo Médio de Reparo* (MTTR) mede quanto tempo, em média, é gasto para restaurar o sistema após uma falha. Combinando MTTF e MTTR, obtém-se a disponibilidade aproximada:

$$AVAIL = \frac{MTTF}{MTTF + MTTR}.$$

Do ponto de vista prático, é comum falar nos “noves” de disponibilidade (por exemplo, 99,9% ou 99,99%), que indicam quanto tempo de parada é aceitável em um determinado período.

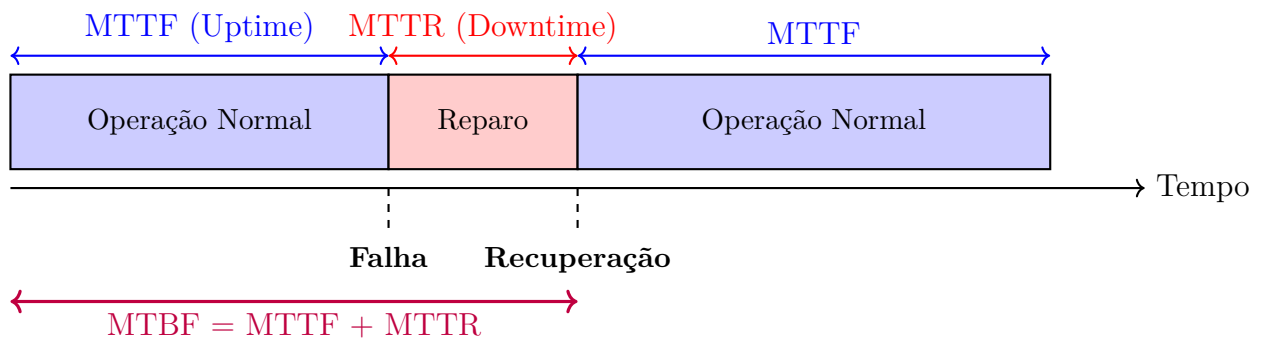


Figura 2.4: Relação métrica entre MTTF, MTTR e MTBF na linha do tempo operacional.

2.8 Visão Baseada em Entradas e Saídas

Outra forma de entender confiabilidade é ver o sistema como um mapeamento de entradas em saídas. Nem todas as entradas possíveis levam a falhas; apenas um subconjunto de entradas (ou sequências de entradas) causa a execução de código defeituoso e, potencialmente, falhas. A frequência com que esse subconjunto é exercitado pelos usuários determina, em grande parte, a confiabilidade percebida.

Usuários diferentes podem ter percepções distintas da confiabilidade do mesmo sistema, justamente porque utilizam subconjuntos de funcionalidades diversos. Um usuário cujo perfil de uso nunca atinge o conjunto de entradas problemáticas pode considerar o sistema “perfeito”, enquanto outro, que depende intensamente dessas funções, experimenta falhas recorrentes.

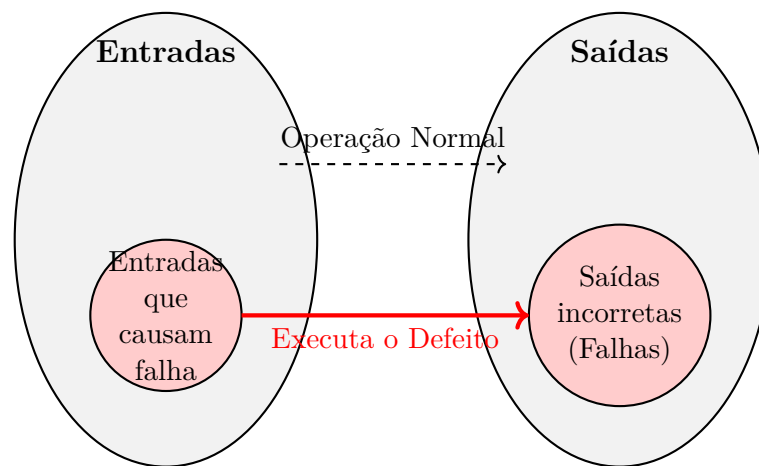


Figura 2.5: Visão de confiabilidade baseada no mapeamento de entradas e saídas.

2.9 Considerações Finais sobre Sommerville

Os conceitos apresentados por Sommerville oferecem uma base formal para discutir confiabilidade, disponibilidade e dependabilidade em sistemas de software. Eles ajudam a esclarecer:

- como erros humanos se transformam (ou não) em falhas observáveis;
- por que nem todo defeito deve ser tratado com a mesma urgência;
- como equilibrar custos de prevenção, detecção e tolerância;
- de que maneira diferentes métricas capturam aspectos distintos do comportamento do sistema.

Essas ideias serão aprofundadas em exercícios, estudos de caso e atividades práticas ao longo da disciplina, reforçando a ligação entre teoria e aplicação em contextos reais.

Capítulo 3

Arquiteturas Tolerantes a Defeitos e Diversidade de Software

Este capítulo apresenta um panorama das arquiteturas tolerantes a defeitos estudadas na disciplina, com base na aula sobre sistemas de proteção, arquiteturas de automonitoramento, programação N-versões e diversidade de software. A ênfase está em compreender *como* a arquitetura contribui para a confiabilidade e quais são os trade-offs de custo, complexidade e disponibilidade associados a cada abordagem.

3.1 Arquiteturas Tolerantes a Defeitos

Arquiteturas tolerantes a defeitos têm como objetivo permitir que o sistema continue operando mesmo quando alguns de seus componentes falham. Em vez de assumir que o software ou o hardware serão perfeitos, essas arquiteturas incorporam redundância, mecanismos de detecção de falhas e estratégias de recuperação.

3.1.1 Visão Geral em Blocos

De forma abstrata, podemos representar uma arquitetura tolerante a defeitos como na Figura 3.1: um conjunto de componentes que recebem entradas, executam processamento redundante e entregam uma saída filtrada, após detecção e eventual mascaramento de falhas.

Nessa visão, o mecanismo de detecção/votação é responsável por identificar discrepâncias entre as saídas dos componentes e, quando possível, selecionar um resultado considerado correto, isolando os canais defeituosos.

Um exemplo moderno é o ecossistema Hadoop, projetado para processamento distribuído de grandes volumes de dados. Em vez de um supercomputador centralizado, utiliza-se um conjunto de máquinas comuns (*commodity hardware*) organizadas em clus-

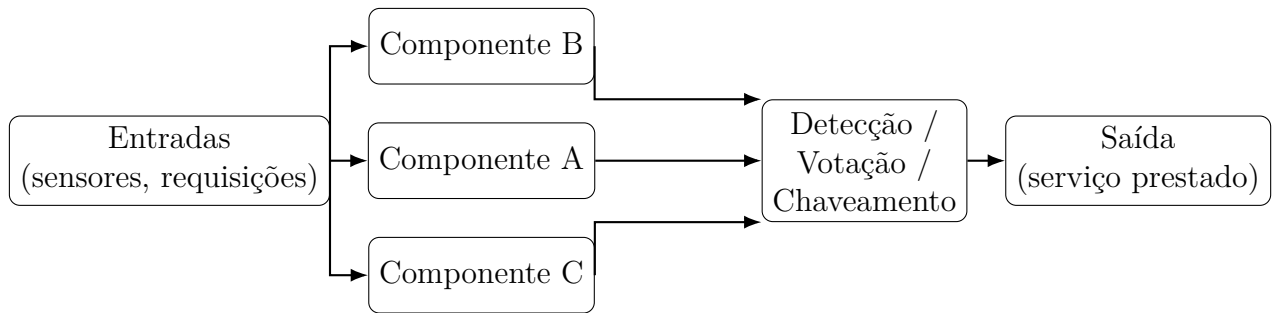


Figura 3.1: Esquema conceitual de uma arquitetura tolerante a defeitos

ter. O sistema de arquivos distribuído HDFS replica blocos de dados em diferentes nós; se uma máquina falha, os dados continuam acessíveis em outras, e o cluster pode se auto-reparar criando novas cópias. A Figura 3.2 resume essa ideia.

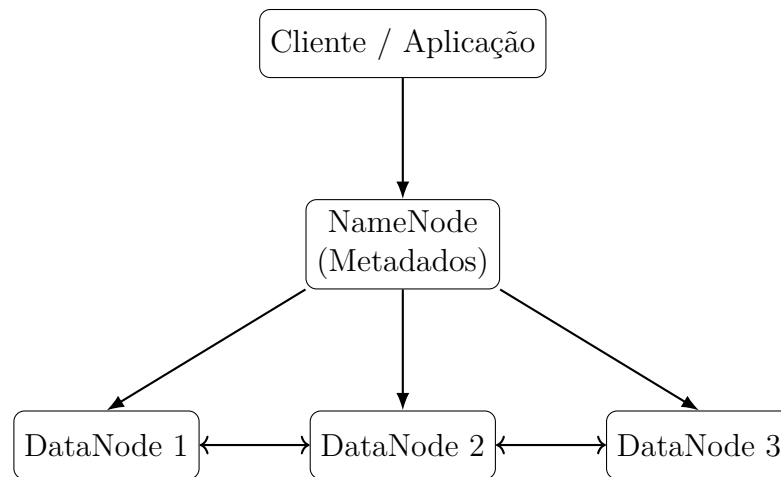


Figura 3.2: Visão simplificada de um cluster Hadoop com replicação de dados

3.2 Sistemas de Proteção

Um sistema de proteção é um sistema especializado, associado a um sistema principal mais complexo, cuja função é garantir que nada catastrófico aconteça. Ele atua como supervisor independente:

- observa o ambiente físico e o comportamento do sistema controlado;
- detecta situações perigosas;
- intervém para levar o sistema a um estado seguro.

Exemplos incluem sistemas de segurança em plantas industriais, trens autônomos e veículos espaciais. No caso de um trem, por exemplo, se o software de controle não desacelera ao ultrapassar um sinal vermelho, o sistema de proteção assume o controle e aciona os freios, independentemente da decisão do sistema principal.

A função essencial do sistema de proteção é implementar apenas a funcionalidade crítica necessária para mover o sistema de um estado perigoso para um estado seguro. Por ser pequeno e simples, é possível investir mais em especificação, verificação e validação, visando probabilidades muito baixas de falha sob demanda.

3.2.1 Diagrama de um Sistema de Proteção

A Figura 3.3 ilustra a relação entre o sistema principal, o ambiente físico e o sistema de proteção:

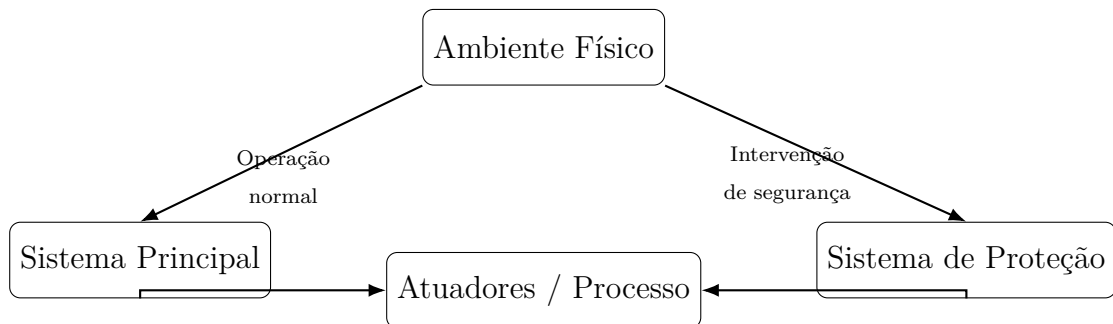


Figura 3.3: Interação entre sistema principal, sistema de proteção e ambiente

O sistema de proteção monitora continuamente o ambiente e o comportamento do sistema principal. Quando detecta uma condição de risco, assume o controle dos atuadores para colocar o sistema em um estado seguro, mesmo que isso implique interromper a operação normal.

3.3 Arquiteturas de Automonitoramento

Na arquitetura de automonitoramento, o próprio sistema monitora sua operação por meio de canais redundantes. A mesma entrada é enviada a dois ou mais canais que executam o mesmo cálculo de forma independente; seus resultados são então comparados.

Se os resultados coincidem, considera-se que a operação foi correta; se divergem, uma falha é detectada e o sistema pode desligar-se, sinalizar erro ou transferir o controle para outro componente. Essa abordagem é particularmente adequada quando a confiabilidade (evitar respostas erradas) é mais importante que a disponibilidade (manter o serviço a qualquer custo) — por exemplo, em sistemas médicos de diagnóstico.

Quando também se exige alta disponibilidade, é possível combinar vários sistemas automonitorados em paralelo, com uma unidade de chaveamento que seleciona a saída considerada válida, permitindo que o serviço continue mesmo na presença de múltiplas falhas internas.

3.3.1 Esquema de Dois Canais em Paralelo

A Figura 3.4 mostra um exemplo de arquitetura de automonitoramento com dois canais redundantes e um comparador:

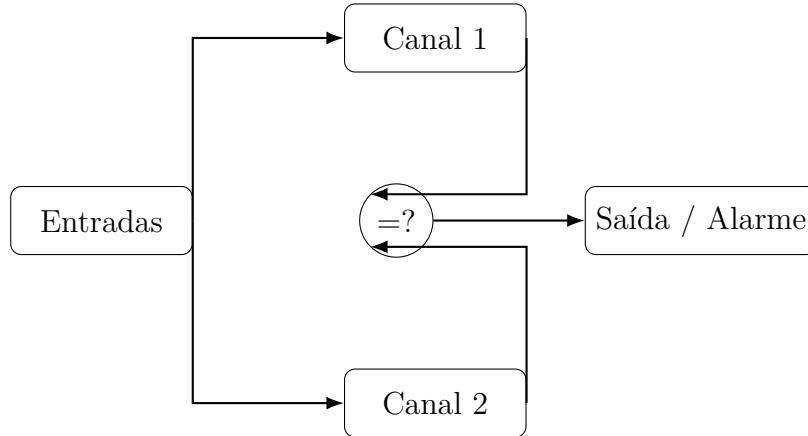


Figura 3.4: Arquitetura de automonitoramento com dois canais redundantes

Se as saídas dos canais divergem, o comparador sinaliza falha e o sistema entra em modo de segurança ou transfere o controle para uma arquitetura alternativa (por exemplo, um sistema de proteção ou um terceiro canal).

3.4 Diversidade de Hardware e de Software

Tanto sistemas de proteção quanto arquiteturas de automonitoramento se beneficiam de diversidade. Se todos os canais forem idênticos, um único defeito de projeto pode levar a falhas simultâneas.

3.4.1 Diversidade de Hardware

A diversidade de hardware busca utilizar processadores, chipsets e implementações físicas de fornecedores distintos, reduzindo a probabilidade de que um defeito de projeto de um componente afete todos os canais ao mesmo tempo.

3.4.2 Diversidade de Software

Já a diversidade de software procura evitar que o mesmo bug esteja presente em todas as versões. Para isso, recomendam-se:

- equipes independentes, sem comunicação direta durante o desenvolvimento;
- diferentes linguagens de programação, ferramentas e métodos de projeto;
- algoritmos distintos para resolver o mesmo problema.

Ainda assim, estudos mostram que a independência nunca é completa: equipes com formação e referências semelhantes tendem a cometer erros parecidos e a interpretar requisitos ambíguos da mesma forma. Diversidade reduz, mas não elimina, o risco de falhas de modo comum.

3.5 Programação N-versões

A programação N-versões é uma técnica de tolerância a defeitos baseada em redundância de software. Partindo da ideia de *Redundância Modular Tripla* (TMR) em hardware, desenvolvem-se N implementações independentes da mesma especificação, que são executadas em paralelo. Um componente votador compara as saídas:

- se a maioria concorda, o resultado é aceito;
- saídas discrepantes são tratadas como defeituosas e podem ser descartadas.

Para tolerar a falha de uma versão, são necessárias pelo menos três implementações. Em teoria, se as versões fossem completamente independentes, a confiabilidade global poderia aumentar drasticamente. Na prática, a existência de erros comuns reduz esse ganho, mas ainda assim sistemas com múltiplos canais tendem a ser significativamente mais confiáveis do que sistemas únicos.

O custo, porém, é elevado: exige múltiplas equipes, ferramentas e implementações. Por isso, a programação N-versões é normalmente reservada a sistemas de missão crítica, como sinalização ferroviária, controle aeronáutico e proteção de reatores nucleares. Em muitos casos, uma alternativa prática é combinar um sistema principal completo com um sistema de proteção ou backup simples, responsável apenas por levar o sistema a um estado seguro.

3.5.1 Diagrama Conceitual de N-versões

A Figura ?? representa uma arquitetura de três versões independentes com um votador:

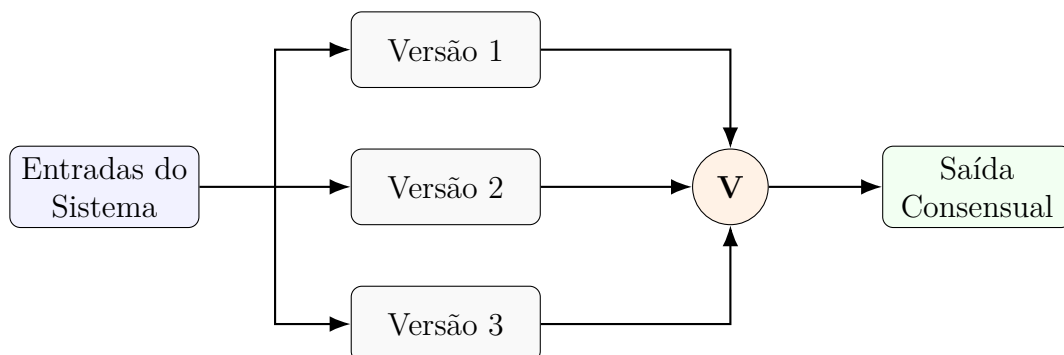


Figura 3.5: Arquitetura de Programação N-versões: redundância para tolerância a falhas.

Nesse tipo de arquitetura, a diversidade entre versões é crucial para que uma única falha não comprometa o resultado global.

3.6 Conclusões sobre Diversidade e Tolerância a Defeitos

De forma geral, arquiteturas tolerantes a defeitos e estratégias de diversidade de software aumentam a confiabilidade de sistemas críticos, mas trazem custos adicionais de desenvolvimento e complexidade de projeto. A decisão de adotá-las deve considerar:

- o risco e o impacto de falhas no contexto de uso;
- os requisitos de confiabilidade e disponibilidade;
- os recursos disponíveis para desenvolvimento, verificação e manutenção.

Essas arquiteturas complementam as técnicas de prevenção, detecção e tolerância a defeitos discutidas anteriormente, fornecendo um arcabouço prático para o projeto de sistemas de alta confiabilidade.

3.7 Hadoop: Tolerância a Defeitos em Escala Planetária

Até este ponto do capítulo, estudamos técnicas arquiteturais que permitem a construção de sistemas que continuam funcionando mesmo diante de falhas de componentes individuais, como redundância, N-versões e sistemas de proteção. Uma das maiores implementações práticas desses princípios em escala industrial é o **Apache Hadoop** — um ecossistema de software de código aberto projetado especificamente para processar e armazenar volumes massivos de dados de forma distribuída, tolerante a falhas e horizontal.

3.7.1 O Que é o Hadoop



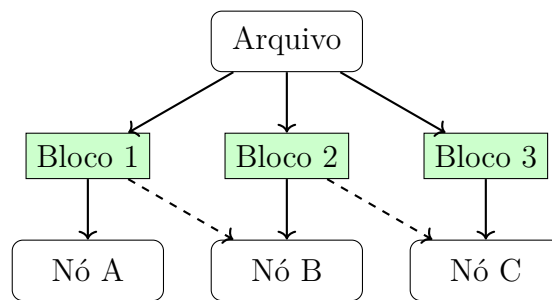
Figura 3.6: Logo Hadoop

O Hadoop nasceu no início dos anos 2000, inspirado por dois artigos publicados pelo Google: o *Google File System* (GFS, 2003) e o modelo de programação *MapReduce* (2004). A questão fundamental que motivou sua criação era: *como processar petabytes de dados de forma eficiente, quando nenhum servidor individual é capaz de conter ou processar tudo?*

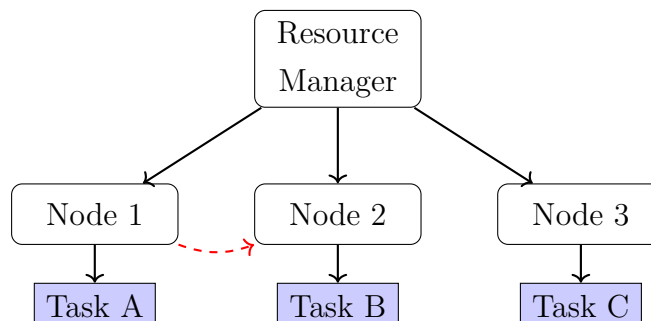
A resposta do Hadoop é elegante em sua simplicidade: **distribua os dados e o processamento por dezenas, centenas ou milhares de servidores comuns** (*commodity hardware*), de baixo custo, e projete o sistema para **tolerar automaticamente a falha de qualquer um deles**.

O ecossistema Hadoop é composto por dois núcleos fundamentais:

- **HDFS (Hadoop Distributed File System)**: O sistema de arquivos distribuído que fragmenta arquivos em blocos (tipicamente de 128 MB) e os replica por múltiplos servidores, garantindo que a perda de nós não resulte em perda de dados.



- **YARN (Yet Another Resource Negotiator)**: O gerenciador de recursos e tarefas que orquestra quais servidores executam cada parte do processamento, redistribuindo trabalho automaticamente quando um nó falha.



3.7.2 Como Funciona o Cluster Hadoop

Um cluster Hadoop típico segue uma arquitetura *master-worker* (mestre-trabalhador) claramente definida:

NameNode (Mestre HDFS): Mantém o *namespace* de todos os arquivos e o mapeamento de quais blocos de dados estão armazenados em quais DataNodes. Não armazena dados diretamente — apenas os metadados. O ponto crítico: **se o NameNode falhar, o cluster perde a capacidade de localizar os dados**, o que historicamente era o maior *single point of failure* do Hadoop (mitigado em versões modernas com o *HDFS High Availability*).

DataNodes (Trabalhadores HDFS): Servidores comuns que armazenam fisicamente os blocos de dados. Cada bloco é replicado por padrão em **3 nós distintos**, preferencialmente em racks diferentes, garantindo que a falha de um servidor ou de um rack inteiro não cause perda de dados.

ResourceManager (Mestre YARN): Responsável por aceitar os jobs de processamento submetidos pelos clientes e alocar recursos (CPU e memória) nos NodeManagers disponíveis, seguindo políticas de escalonamento.

NodeManagers (Trabalhadores YARN): Executam as tarefas de processamento em seus respectivos servidores e reportam periodicamente seu estado ao ResourceManager. Se um NodeManager silencia (crash), o ResourceManager redistribui suas tarefas para outros nós sem intervenção humana.

A Figura ?? ilustra a topologia de um cluster Hadoop típico, mostrando como o processamento e os dados se distribuem globalmente.

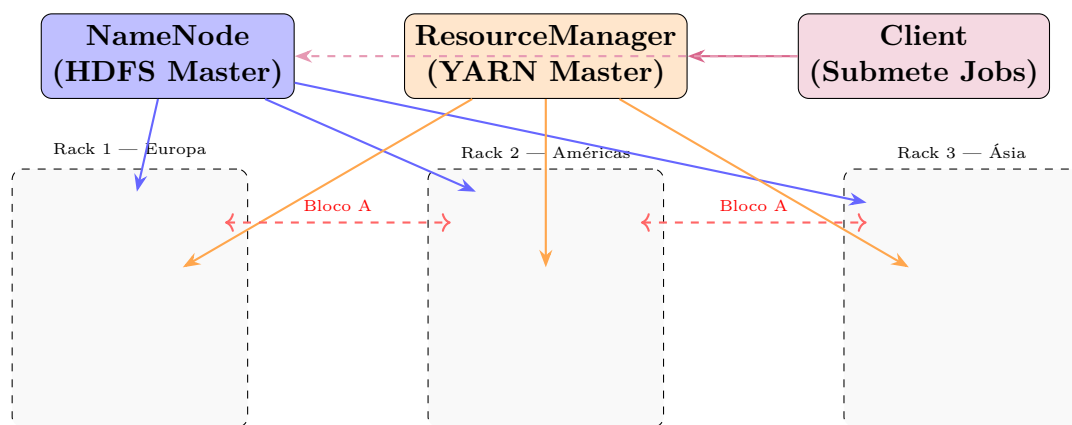


Figura 3.7: Topologia de um Cluster Hadoop distribuído globalmente.

3.7.3 Tolerância a Defeitos no Hadoop

A tolerância a defeitos no Hadoop é implementada em múltiplas camadas, refletindo diretamente os princípios estudados neste capítulo:

1. **Replicação de Dados (Redundância):** Cada bloco de dados é automaticamente replicado em 3 nós distintos (por padrão), priorizando nós em racks diferentes. Isso garante que mesmo a falha de um rack inteiro não resulte em perda de dados. O HDFS detecta a falha de um DataNode via *heartbeats* (batimentos cardíacos) periódicos e inicia automaticamente a re-replicação dos blocos que ficaram abaixo do fator de replicação mínimo.
2. **Reexecução Automática de Tarefas:** Quando o YARN detecta que um Node-Manager parou de responder, as tarefas em execução nesse nó são marcadas como falhas e automaticamente reagendadas em outros nós disponíveis, sem necessidade de intervenção humana.
3. **Especulação (Slow-Task Detection):** O YARN monitora o progresso de todas as tarefas paralelas. Se uma tarefa for detectada como significativamente mais lenta que suas paralelas (possivelmente por hardware degradado), ele inicia especulativamente uma *cópia* desta tarefa em outro nó, usando o resultado da que terminar primeiro.
4. **Isolamento de Defeitos:** Como o processamento é dividido em unidades independentes (mappers e reducers), a falha de uma unidade não contamina as demais. O princípio é análogo à programação N-versões: cada unidade de trabalho é isolada e pode ser reexecutada independentemente.

O Hadoop é, portanto, um exemplo concreto de como os princípios teóricos de tolerância a defeitos — redundância, detecção automática, recuperação sem intervenção humana e isolamento de falhas — se materializam em uma plataforma utilizada por empresas como o Yahoo, Facebook, LinkedIn e centenas de outras organizações para processar dados em escala que seria impossível em sistemas sem essas garantias arquiteturais.

Capítulo 4

Monitoração Agregada: Princípios do Google SRE e Métricas de Confiabilidade

A monitoração é um dos pilares fundamentais da Engenharia de Confiabilidade de Sites (SRE). No Google, a máxima é simples: se você não está medindo, você não sabe como seu sistema está operando. Este capítulo explora a transição da monitoração tradicional para a monitoração baseada em métricas agregadas, focando na experiência do usuário e na gestão de riscos.

4.1 A Filosofia da Medição no Google SRE

Por que medimos? Em sistemas complexos e de larga escala, o desempenho e a saúde do sistema não podem ser avaliados por simples intuição ou observação esporádica. A monitoração cumpre papéis vitais que vão além da simples detecção de falhas:

- **Observabilidade:** Permitir que engenheiros entendam o estado interno do sistema a partir de suas saídas externas.
- **Ciclos de Feedback:** Sem dados, a melhoria contínua (PDCA) é impossível. Métricas permitem validar se uma mudança no código ou na infraestrutura surtiu o efeito desejado.
- **Definição de Metas Objetivas:** Permite a criação de contratos formais como SLAs e metas internas como SLOs.
- **Alinhamento de Incentivos:** Fornece uma linguagem comum entre desenvolvedores (que buscam velocidade) e operadores (que buscam estabilidade).

No Google, busca-se definir uma **métrica objetiva** para representar a propriedade que queremos otimizar. Se o risco não for traduzido em métrica, ele não pode ser gerenciado.

4.2 Impactos das Falhas de Serviço

Uma falha de serviço não é apenas um "bug" no código; ela tem repercussões que afetam diversas dimensões da organização. A Figura 4.1 ilustra essa cadeia de consequências.

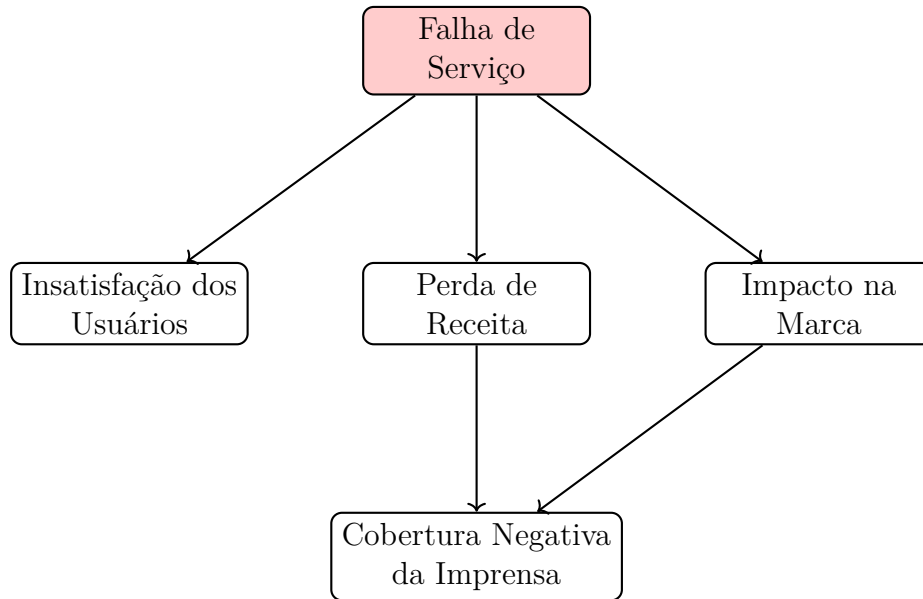


Figura 4.1: Impactos multidimensionais de uma falha de serviço

Para mitigar esses riscos, o SRE foca em reduzir o **downtime não planejado**.

4.3 Downtime Não Planejado e Disponibilidade

Downtime não planejado corresponde aos períodos em que o serviço fica indisponível sem aviso prévio. Diferencia-se das manutenções programadas, que são comunicadas e, em geral, não consomem o "orçamento de erro" da equipe.

4.3.1 Cálculo de Disponibilidade Baseada em Tempo

Tradicionalmente, a disponibilidade é calculada pela proporção de tempo em que o sistema está operacional (uptime) em relação ao tempo total:

$$\text{Disponibilidade} = \frac{\text{Tempo de Uptime}}{\text{Tempo Total}} \quad (4.1)$$

A Figura 4.2 representa graficamente essa alternância ao longo do tempo.

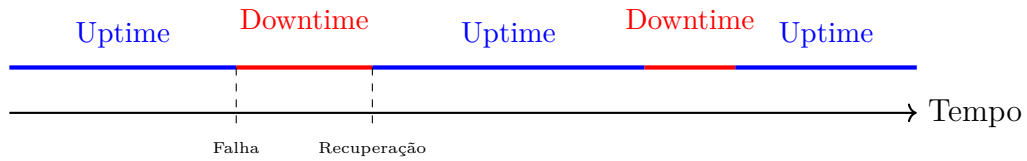


Figura 4.2: Linha do tempo representando períodos de Uptime e Downtime

4.3.2 A Escala dos Noves

A meta de disponibilidade é frequentemente expressa pelo número de dígitos "9". Cada dígito adicional representa uma redução drástica no tempo de inatividade tolerável:

Nível	Percentual	Downtime Anual Máximo
3 novos	99,9%	8h 45min 57s
4 novos	99,99%	52min 35s
5 novos	99,999%	5min 15s

Tabela 4.1: O impacto de cada "nove" adicional na disponibilidade

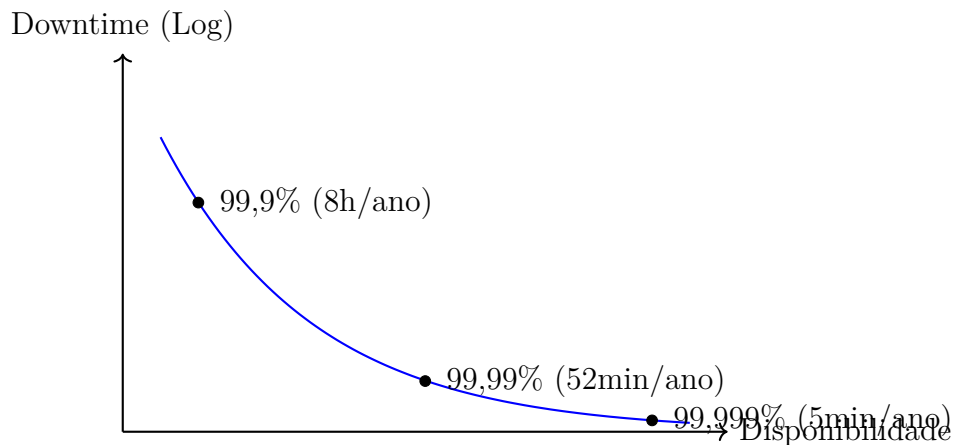


Figura 4.3: Relação entre percentual de disponibilidade e tempo de inatividade

4.4 Confiabilidade em Sistemas Distribuídos

A métrica baseada em tempo (uptime/total) funciona bem para um servidor solitário. No entanto, o Google opera sistemas globais com milhares de servidores em dezenas de regiões. Nestes cenários, o sistema quase nunca está 100% fora do ar, mas frequentemente está operando com **capacidade reduzida** ou falhas em uma região específica.

4.4.1 O Problema da Falha Localizada

Imagine um sistema com datacenters na América, Europa e Ásia. Se o datacenter da Europa falhar, o serviço ainda estará disponível para os usuários das outras regiões. A Figura 4.4 ilustra essa situação.

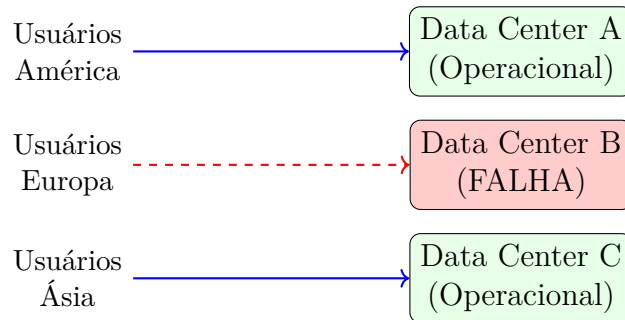


Figura 4.4: Isolamento regional de falhas em sistemas distribuídos

Nesse caso, o uptime "total" é ambíguo. Por isso, o Google utiliza a **Disponibilidade Baseada em Requisições**.

4.5 Disponibilidade Baseada em Requisições

Esta métrica foca na experiência real do usuário: "Minha tentativa de usar o sistema funcionou ou falhou?". Em vez de medir segundos de inatividade, medimos o sucesso de cada interação individual.

$$\text{Disponibilidade} = \frac{\text{Requisições Bem Sucedidas}}{\text{Total de Requisições}} \quad (4.2)$$

Esta visão é muito mais granular e permite calcular a saúde do sistema mesmo em situações de degradação parcial.

4.5.1 Exemplo de Cálculo de Orçamento de Erro

Considere um serviço que processa média de 2,5 milhões de requisições diárias com uma meta de 99,99%.

- Total: 2.500.000 requisições.
- Erro Máximo Permitido ($0,01\% \times 2.500.000$): 250 erros.

Isso significa que o sistema pode falhar até 250 vezes em um dia e ainda assim cumprir seu objetivo de confiabilidade.

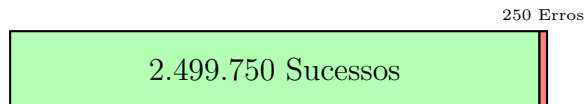


Figura 4.5: Visualização do orçamento de erros em relação ao volume de tráfego

4.6 Monitoração de APIs REST e Protocolo HTTP

Como a maioria dos serviços modernos é exposta através de APIs REST, a saúde do serviço é medida através das respostas HTTP. Para um SRE, entender o que cada código significa é crucial para classificar o sucesso ou a falha.

4.6.1 A Requisição HTTP

Uma requisição é o pedido do cliente ao servidor. Ela contém:

- **Método:** A ação desejada (GET, POST, etc).
- **Recurso:** A URL do objeto a ser manipulado.
- **Corpo:** Dados enviados (JSON, XML).

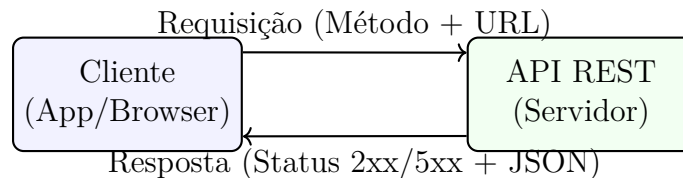


Figura 4.6: Fluxo de comunicação em uma API REST

4.6.2 Métodos HTTP Principais

Para fins de confiabilidade, os métodos possuem propriedades diferentes:

1. **GET:** Recupera dados. É seguro e *idempotente* (pode ser repetido sem mudar o estado). Falhas em GET são frequentes alvos de retentativas automáticas.
2. **POST:** Cria novos recursos. **Não** é idempotente. Falhas em POST exigem cuidado especial, pois repetir uma criação pode gerar duplicatas.
3. **PUT:** Atualiza um recurso existente. É idempotente.
4. **DELETE:** Remove um recurso. É idempotente.

4.7 Classificação de Códigos de Status HTTP

O SRE deve saber distinguir se uma falha é culpa do cliente ou do servidor.

Faixa 1xx (Informacional) : Raro em monitoração de alto nível. Indica que o servidor recebeu o cabeçalho e está aguardando o corpo da mensagem. Exemplo: 100 Continue.

Faixa 2xx (Sucesso) : Indica que a operação foi concluída. Note que 200 (OK), 201 (Created) e 204 (No Content) são todos "sucessos" para a métrica de disponibilidade.

Faixa 3xx (Redirecionamento) : O recurso mudou de lugar. Geralmente tratado pelo cliente automaticamente. Não é falha de disponibilidade, mas pode indicar má arquitetura se excessivo.

Faixa 4xx (Erro do Cliente) : O cliente pediu algo impossível ou inválido (ex: 404 Not Found, 400 Bad Request). **Para o SRE, isso NÃO conta como indisponibilidade**, pois o servidor funcionou corretamente ao recusar a requisição inválida.

Faixa 5xx (Erro do Servidor) : O servidor falhou ao processar uma requisição válida (ex: 500 Internal Error, 502 Bad Gateway). Isso **CONTA** como indisponibilidade e consome o orçamento de erro.

4.8 Estudo de Caso: Caldeira Industrial Siderúrgica

Para ilustrar esses conceitos, examinaremos um sistema de monitoramento de uma caldeira industrial. Imaginemos que um técnico de QA realizou 6 testes manuais no endpoint `/caldeira/status` e `/caldeira/manutencao`.

4.8.1 Logs de Execução

1. **GET** `/status` → 200 OK. Sucesso.
2. **POST** `/manutencao` (corpo vazio) → 400 Bad Request. O sistema barrou o dado inválido. Sucesso da lógica de validação.
3. **POST** `/manutencao` (dados OK) → 201 Created. Sucesso.
4. **GET** `/manutencao/999` → 404 Not Found. O registro não existe. Resposta correta do sistema.
5. **DELETE** `/manutencao/52` → 204 No Content. Sucesso.

6. **DELETE /manutencao/60** (durante queda do banco) → 500 **Internal Error**.
Falha interna.

4.8.2 Cálculo da Disponibilidade Observada

Se olharmos apenas para os sucessos explícitos (2xx), temos 3 sucessos de 6 requisições (50%). No entanto, a análise de SRE é mais refinada:

Código	Classificação	Conta para Disponibilidade?
200, 201, 204	Sucesso	Sim
400, 404	Erro de Cliente	Sim (Sistema funcionou)
500	Erro de Servidor	Não (Sinal de indisponibilidade)

Disponibilidade Real: $\frac{5}{6} \approx 83,33\%$.

Apenas o erro 500 foi uma falha de engenharia/infraestrutura que o usuário não deveria ver.

4.9 Service Level Objectives (SLO) e Error Budgets

A monitoração agregada culmina no conceito de **SLO**. O SLO é o valor alvo para um nível de serviço (ex: "disponibilidade de 99,9%").

O segredo do SRE é o **Error Budget** (Orçamento de Erro). Ele é definido como $1 - SLO$. Se sua meta é 99,9%, seu orçamento é 0,1%. Este orçamento pode ser gasto com:

- Lançamento de novas funcionalidades (que trazem risco).
- Falhas imprevistas.
- Manutenções.

Se o orçamento acaba antes do final do período, a equipe para de lançar novas funcionalidades e foca exclusivamente em confiabilidade até que a métrica se estabilize. Isso resolve o conflito histórico entre Dev e Ops através de uma regra matemática objetiva.

4.10 Exercícios de Revisão

1. Explique por que um erro 404 **Not Found** não deve ser contabilizado como indisponibilidade do sistema de acordo com os princípios de SRE.
2. Um sistema recebeu 10 mil requisições em 1 hora. Houve 50 erros 500 e 100 erros 403. Qual a disponibilidade do sistema nessa hora?

3. Desenhe a linha do tempo de um sistema que opera por 20 minutos, cai por 5 minutos, e volta a operar por mais 15 minutos. Calcule a disponibilidade baseada em tempo para este período total de 40 minutos.
4. Qual a diferença fundamental entre métricas baseadas em tempo e métricas baseadas em requisições para sistemas distribuídos?

Capítulo 5

Métricas de Confiabilidade no Modelo Google SRE

A medição é o pilar sobre o qual se sustenta toda a disciplina de Site Reliability Engineering (SRE). Sem métricas objetivas, a confiabilidade permanece no reino da intuição e da percepção subjetiva, o que é insuficiente para gerenciar sistemas complexos em escala global. Este capítulo explora como o Google quantifica riscos e desempenho, transformando abstrações operacionais em indicadores matemáticos que guiam decisões de negócio e de engenharia.

5.1 A Filosofia da Medição e a Gestão de Riscos

No Google, a prática padrão para qualquer serviço é a definição de métricas que representem fielmente as propriedades que se deseja otimizar. A medição não é um fim em si mesma, mas uma ferramenta para avaliar o desempenho atual, monitorar a eficácia de melhorias e, crucialmente, identificar degradações sutis antes que elas se tornem catastróficas.

O desafio reside na redução da complexidade de um sistema distribuído a indicadores que sejam, ao mesmo tempo, simples de entender e tecnicamente precisos. Quando falamos em "Métricas Google SRE", estamos falando de uma linguagem comum que permite que desenvolvedores (Dev) e engenheiros de confiabilidade (SRE) discutam o estado do sistema sem ambiguidades.

5.1.1 Impactos das Falhas de Serviço

Uma falha de serviço raramente é um evento isolado sem consequências externas. A indisponibilidade de um sistema crítico desencadeia uma reação em cadeia que afeta múltiplas dimensões da organização. No modelo SRE, mapeamos esses impactos para entender a

severidade do risco.

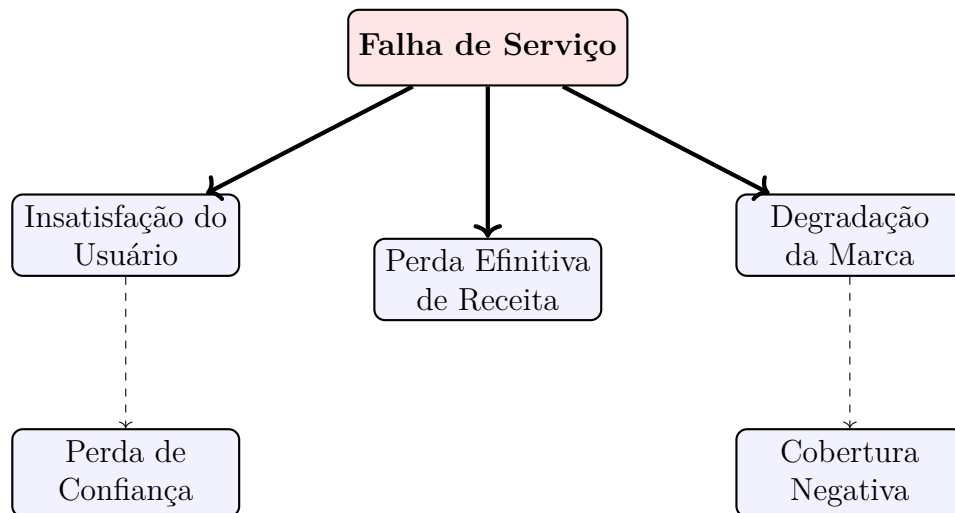


Figura 5.1: Mapa de consequências de falhas de serviço conforme princípios SRE.

Entre os impactos fundamentais, destacamos:

- **Satisfação do Usuário:** O dano mais imediato. Um usuário que não consegue realizar uma tarefa no sistema torna-se propenso a migrar para um concorrente.
- **Perda de Receita:** Em sistemas de e-commerce ou publicidade, cada segundo de indisponibilidade traduz-se diretamente em perda financeira irre recuperável.
- **Valor da Marca:** A reputação de uma empresa é construída ao longo de anos e pode ser severamente abalada por um incidente de grande escala que ganhe as manchetes da imprensa.

5.2 Downtime não Planejado como Métrica Central

Embora existam muitos tipos de riscos, o Google escolheu focar em uma métrica unificadora: o **downtime não planejado**. Esta escolha deve-se à sua capacidade de ser padronizada e comparada entre diferentes tipos de serviços, desde APIs de infraestrutura até aplicativos voltados ao consumidor final.

5.2.1 Definição e Importância

O downtime não planejado corresponde a qualquer período em que o serviço está indisponível sem que tenha havido uma comunicação ou programação prévia. Diferente das janelas de manutenção (downtime planejado), o downtime não planejado é o que realmente consome o orçamento de erro e afeta a percepção do usuário sobre a confiabilidade do sistema.

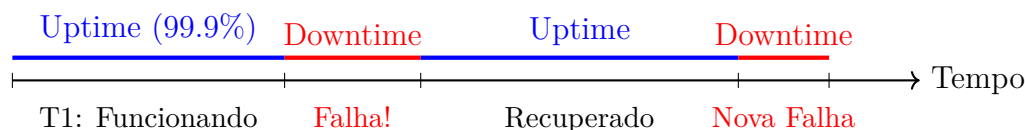


Figura 5.2: Visualização temporal de ciclos de Uptime e Downtime.

5.3 Disponibilidade: A Matemática dos Noves

A disponibilidade é expressa como o percentual de tempo em que um serviço está operacional. No mercado, essa métrica é frequentemente descrita pelo número de "noves" (ex: "cinco noves").

Tabela 5.1: Escala SRE de Disponibilidade e Tempo de Inatividade Anual

Disponibilidade	Downtime Anual	Classificação
99%	3,65 dias	Padrão Básico
99,9%	8h 45min	"Três Noves" (SaaS)
99,99%	52min 35s	"Quatro Noves" (Crítico)
99,999%	5min 15s	"Cinco Noves" (Infra)
99,9999%	31,5 segundos	"Seis Noves" (Ultra)

É fundamental compreender que cada "nove" adicional aumenta a dificuldade de engenharia em uma ordem de magnitude, enquanto reduz o tempo de inatividade permitido drasticamente. Alcançar 99,999% exige automação total da resposta a incidentes, pois o tempo humano de reação (detecção, triagem e resolução) frequentemente excede os 5 minutos anuais permitidos.

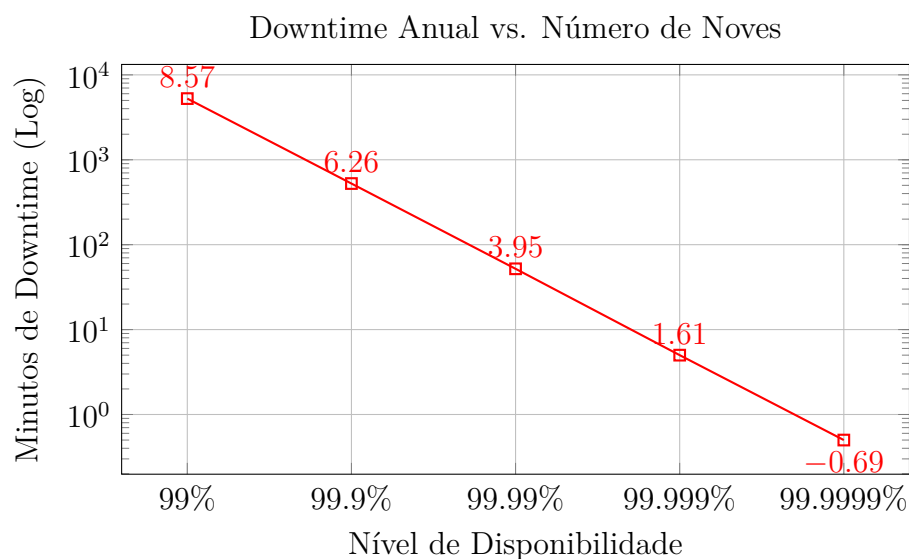


Figura 5.3: Gráfico logarítmico demonstrativo da redução drástica do orçamento de tempo conforme aumentamos os "noves".

5.4 Limitações do Modelo de Disponibilidade Baseada em Tempo

Em sistemas modernos e geograficamente distribuídos, a métrica de disponibilidade baseada puramente em tempo (Uptime/Total) torna-se falha. Imagine um serviço global rodando em 10 data centers. Se um data center na Europa falha, mas os outros na América e Ásia continuam atendendo usuários, o sistema está "online" ou "offline"?

Medir isso como "tempo fora do ar" não captura a realidade da experiência do usuário europeu, nem o sucesso dos usuários americanos. Em sistemas distribuídos, raramente um sistema está 100% inoperante; ele está, quase sempre, "parcialmente degradado".

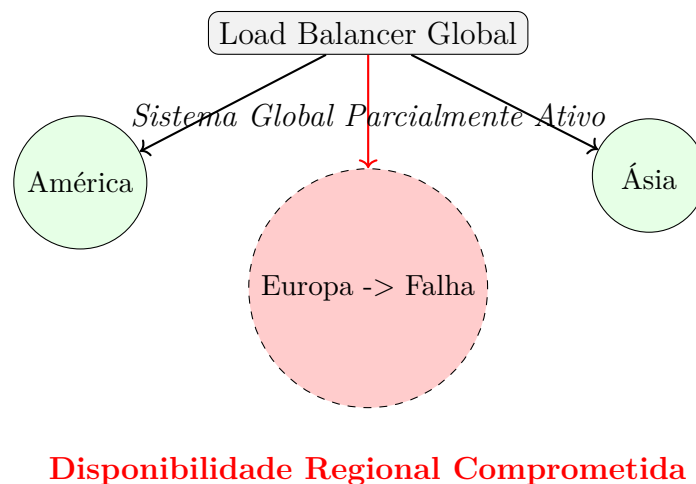


Figura 5.4: Cenário de falha regional em arquitetura distribuída.

5.5 APIs REST e Requisições: A Unidade Fundamental de Sucesso

Para resolver o problema da medição em sistemas distribuídos, o Google e a comunidade SRE adotaram a **Disponibilidade Baseada em Requisições**. Sob esta ótica, não perguntamos por quanto tempo o servidor ficou ligado, mas sim: "Das requisições que chegaram, quantas foram bem-sucedidas?".

5.5.1 O Protocolo HTTP e a Confiabilidade

A comunicação em sistemas modernos baseia-se quase integralmente no protocolo HTTP dentro do estilo arquitetural REST. Cada interação é uma Requisição (*Request*) seguida de uma Resposta (*Response*).

Para o SRE, é crucial entender as propriedades dos métodos HTTP:

- **Idempotência:** Métodos como GET, PUT e DELETE são idempotentes. Isso significa que, se uma requisição falhar devido a um erro de rede, o cliente pode repetir a operação com segurança, pois o estado final do servidor será o mesmo.
- **Segurança:** O método GET é considerado seguro por não alterar o estado do recurso, sendo ideal para monitoramento passivo de saúde (*Health Checks*).
- **Risco de POST:** O método POST **não** é idempotente. Falhas em POST são as mais difíceis de lidar em termos de confiabilidade, pois não sabemos se o recurso foi criado parcialmente antes da falha.

5.6 Taxonomia SRE de Status Codes HTTP

O código de status HTTP retornado pelo servidor é a fonte da verdade para a métrica de disponibilidade. O SRE classifica esses códigos em faixas para determinar se o sistema está saudável.

Faixa 2xx (Sucesso) : Códigos como 200 OK, 201 Created e 204 No Content indicam que a lógica de negócio foi executada e o servidor cumpriu seu papel. São contabilizados como sucessos.

Faixa 4xx (Erro do Cliente) : Inclui 400 Bad Request, 401 Unauthorized e 404 Not Found. **Crucial:** Para o Google SRE, erros 4xx **não** são falhas de disponibilidade. O servidor funcionou perfeitamente ao recusar uma requisição inválida ou proibida. Considerar 404 como erro de servidor levaria a métricas falsas se um atacante fizesse milhares de acessos a URLs inexistentes.

Faixa 5xx (Erro do Servidor) : Códigos como 500 Internal Server Error e 503 Service Unavailable são as verdadeiras falhas. Elas indicam um "bug", queda de banco de dados ou exaustão de recursos. Estes códigos consomem diretamente o orçamento de erro.

5.7 Cálculo de Disponibilidade SRE: A Fórmula Fundamental

A disponibilidade SRE é calculada como a razão entre as operações bem-sucedidas e o total de operações válidas:

$$D_{SRE} = \frac{\sum \text{Requisições bem-sucedidas (Status 2xx)}}{\sum \text{Total de Requisições Válidas (Sucessos + Status 5xx)}} \quad (5.1)$$

Note que os erros 4xx são excluídos tanto do numerador quanto do denominador em análises rigorosas de infraestrutura, ou incluídos no denominador apenas se forem considerados "requisições válidas que o sistema tratou com sucesso".

5.7.1 Exemplo de Grande Escala

Suponha um serviço de autenticação que recebeu 100 milhões de requisições em um mês. O SLO definido é de 99,95%. O orçamento de erro permitido é de 0,05%, o que equivale a 50.000 falhas permitidas por mês.

Se o monitoramento detectar 45.000 erros 5xx, o sistema ainda está dentro do SLO (99,955%). No entanto, se ocorrer um pico de instabilidade que gere mais 10.000 erros, o orçamento é estourado e ações imediatas de congelamento de *deploys* entram em vigor.

5.8 SLO e Error Budget: O Equilíbrio entre Inovação e Estabilidade

O **SLO (Service Level Objective)** é o alvo numérico para o nível de serviço. O **Error Budget (Orçamento de Erro)** é o espaço de risco entre a perfeição (100%) e o SLO.

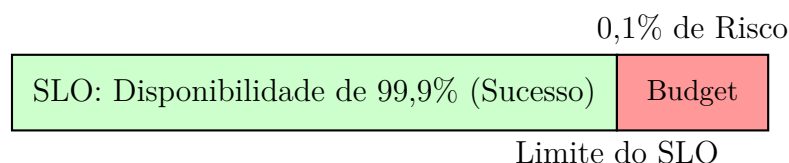


Figura 5.5: Representação visual do orçamento de erro.

Este orçamento remove a subjetividade do conflito Dev vs Ops. Se houver orçamento disponível, os desenvolvedores podem lançar novas versões livremente. Se o orçamento acabar, a estabilidade torna-se a única prioridade. É uma política baseada em dados que alinha os incentivos de todas as equipes.

5.9 Estudo de Caso Avançado: A Caldeira Industrial II

Considere a operação de uma planta siderúrgica onde as caldeiras são controladas por uma API centralizada. Para o QA e o SRE, a análise de um lote de logs é vital.

Logs de Auditoria (1 hora):

1. GET /caldeira/1/temperatura → 200 OK (Sucesso)
2. POST /caldeira/1/ignicao → 500 Error (Banco de dados offline) - **Falha CRÍTICA**
3. GET /caldeira/1/status → 200 OK (Sucesso)
4. PUT /caldeira/1/limite → 403 Forbidden (Token expirado) - **Sistema estável**
5. GET /caldeira/99/status → 404 Not Found (Caldeira inexistente) - **Sistema estável**
6. POST /caldeira/1/limite → 503 Unavailable (Servidor sobrecarregado) - **Indisponibilidade**

Análise de Dados: No total de 6 requisições, tivemos 2 erros da classe 5xx (#2 e #6). Os erros 4xx (#4 e #5) não indicam falha do servidor, mas sim do uso. A disponibilidade percebida pelo SRE para este período seria:

$$D = \frac{4}{6} \approx 66,6\%$$

Este valor está drasticamente abaixo de qualquer SLO comercial aceitável, indicando a necessidade de uma análise de causa-raiz (*Post-mortem*) para a queda do banco e a sobrecarga observada.

5.10 Exercícios de Aprofundamento

1. Por que a métrica de "noves" é considerada logarítmica em termos de esforço de engenharia?
2. Explique o conceito de "Idempotência" e sua importância para a recuperação automática de falhas (Self-healing).
3. Uma empresa possui um SLO de 99,99. Ela realizou 1 milhão de requisições e observou 80 erros 500 e 200 erros 404. Ela cumpriu o SLO? Justifique com cálculos.
4. Diferencie SLO de SLA (Service Level Agreement).
5. Como um SRE deve reagir caso o Error Budget de um serviço seja consumido em apenas uma semana de um trimestre?

6. Desenhe um diagrama de fluxo mostrando o que acontece quando uma requisição retorna um erro 502 Bad Gateway em um ambiente com Load Balancer.
7. Proponha uma estratégia para reduzir o MTTR (Mean Time To Repair) em um sistema que atingiu apenas 99% de disponibilidade anual.
8. Qual o papel dos Health Checks na manutenção da disponibilidade baseada em requisições?

Capítulo 6

Estudo de Caso Prático: Análise de Logs de Servidor Web

Para consolidar os conceitos matemáticos e metodológicos da Engenharia de Confiabilidade apresentados nos capítulos anteriores, este capítulo detalha um estudo de caso prático. A análise de registros de acesso (*logs*) é uma das atividades mais cotidianas de um engenheiro de confiabilidade (SRE) na busca por entender o comportamento real do sistema, identificar gargalos ocultos e calcular as métricas exatas de disponibilidade.

6.1 Contexto e Origem dos Dados

Qualquer servidor web moderno (como Nginx, Apache ou proxies reversos) registra cada interação HTTP que processa. Esses registros são armazenados em arquivos de texto contínuos, conhecidos como *access logs*.

Neste estudo de caso, analisamos um lote de logs contendo pouco mais de 10.000 requisições a um servidor. A estrutura de cada linha do log obedece a um formato comum no mercado, contendo as seguintes informações na sequência:

- **ID da requisição:** Um identificador único autoincremental.
- **Endereço IP do cliente:** Quem realizou o acesso.
- **Data e horário:** Momento exato com fuso horário.
- **Método HTTP e URL:** A ação solicitada (ex: `GET /imagem.png`).
- **Protocolo:** Versão do HTTP utilizada.
- **Código de status:** A resposta do servidor (ex: 200, 404, 502).
- **Tamanho da resposta:** O número de bytes transferidos.
- **Referer:** A página de origem que encaminhou o usuário.
- **User Agent:** O navegador ou robô (bot) utilizado pelo cliente.
- **Latência:** O tempo total (em milissegundos) gasto pelo servidor para processar a requisição e começar a enviar a resposta.

O domínio desta estrutura é fundamental, pois os logs chegam como *strings* brutas de texto e precisam ser convertidos em dados estruturados (inteiros, flutuantes e datas) para análise estatística.

6.1.1 Revisão: Distribuições Simétricas, Assimétricas e Boxplot

Antes de mergulhar nos dados do log, é imprescindível revisitar os conceitos estatísticos que fundamentam a interpretação dos resultados. Quando plotamos a frequência de uma variável contínua (como a latência), obtemos um **histograma**, cuja forma revela muito sobre o comportamento do sistema.

Distribuição Simétrica (Normal)

Uma distribuição é **simétrica** quando os dados se distribuem uniformemente em torno da média, formando a clássica forma de “sino” (gaussiana). Nesse caso, a **média**, a **mediana** e a **moda** coincidem no mesmo ponto central. Exemplos de fenômenos simétricos incluem a altura de uma população adulta e erros de medição em instrumentos calibrados.

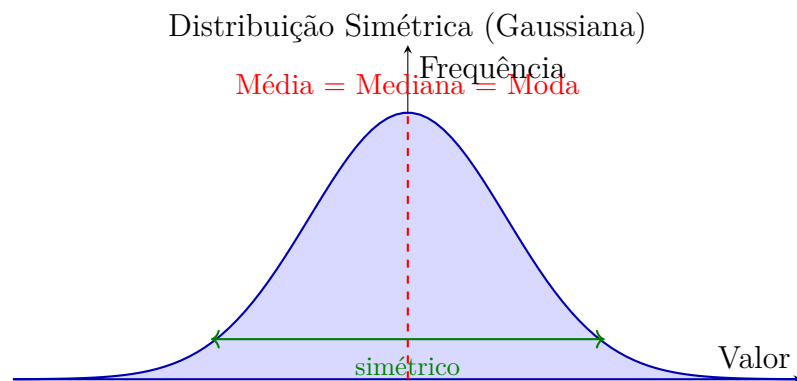


Figura 6.1: Uma distribuição simétrica ideal: a curva é espelhada ao redor da média μ .

Distribuição Assimétrica (Skewed)

Quando os dados **não** se distribuem simetricamente, dizemos que há **assimetria** (*skewness*). Existem dois tipos:

- **Assimetria Positiva (cauda à direita):** A maioria dos valores concentra-se à esquerda, com poucos valores extremamente altos puxando a cauda para a direita. A média é deslocada para a direita da mediana. *Este é o perfil típico de dados de latência*, onde a maioria das requisições é rápida, mas algumas poucas são muito lentas.

- **Assimetria Negativa (cauda à esquerda):** Ocorre ao contrário: poucos valores extremamente baixos puxam a cauda para a esquerda. A média fica à esquerda da mediana.

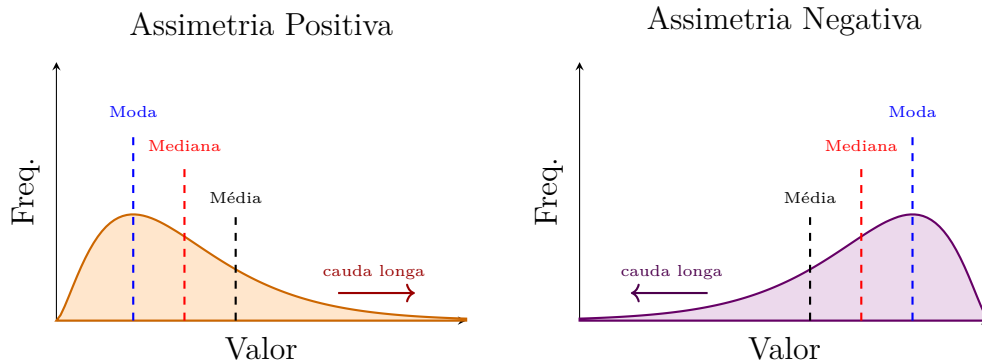


Figura 6.2: Comparação entre distribuições com assimetria positiva (cauda à direita, típica de latência) e negativa (cauda à esquerda).

Na prática de SRE, a distribuição assimétrica positiva é onipresente: a maioria dos pedidos HTTP é respondida rapidamente, mas alguns poucos pedidos “empacam” em filas de banco de dados, falhas de rede ou saturação de CPU, criando a cauda direita. É exatamente por isso que a **média** não é uma boa métrica para latência — ela é puxada pela cauda e superestima o tempo real que a maioria dos usuários percebe.

A Distribuição de Latência e os Percentis P90, P98 e P99

Dados reais de latência de servidores web seguem, na imensa maioria dos casos, uma distribuição **assimétrica à direita**. A Figura 6.3 ilustra este comportamento: o bloco denso de requisições rápidas (entre 100 e 300 ms) forma o pico à esquerda, enquanto uma cauda cada vez mais fina se estende para a direita, representando as poucas requisições que demoram significativamente mais.

Os três percentis marcados na figura têm papéis distintos em acordos de nível de serviço:

- **P90 (≈ 300 ms):** 90% das requisições são atendidas em até 300 ms. É o limiar mais comum para SLOs de latência em serviços internos e APIs corporativas.
- **P98 (≈ 550 ms):** Apenas 2% das requisições ultrapassam este valor. O P98 começa a revelar gargalos reais, como consultas SQL sem índice ou payloads excessivamente grandes.
- **P99 (≈ 800 ms):** O “1% pior caso”. Serviços voltados ao consumidor final (e-commerce, streaming) frequentemente definem SLOs neste nível. Melhorar o P99 exige investigar os *outliers* mais extremos e, geralmente, envolve otimizações arquiteturais profundas (cache, circuit breakers, retries com backoff).

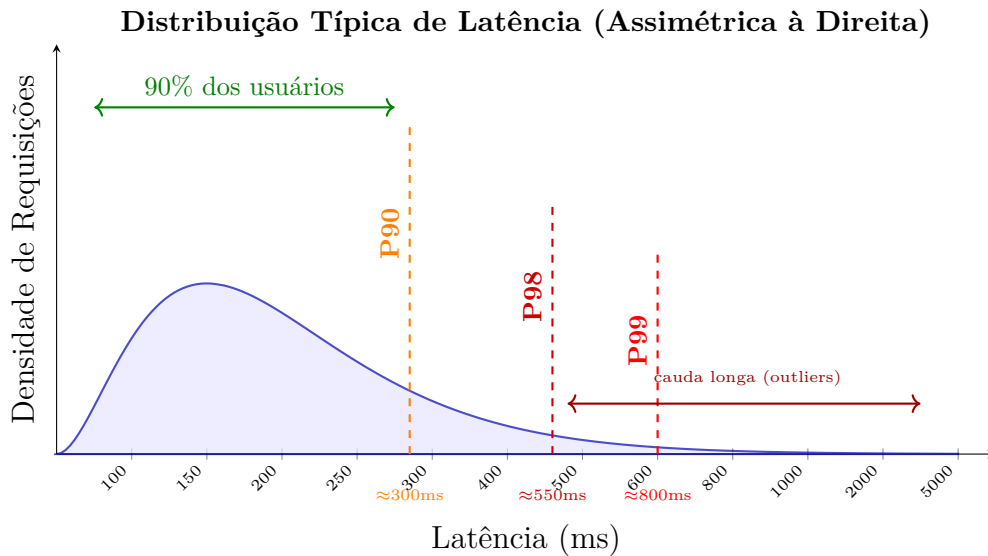


Figura 6.3: Distribuição típica de latência de um servidor Web. As linhas verticais tracejadas marcam os percentis 90, 98 e 99! A grande maioria das requisições é rápida, mas a cauda direita contém os casos problemáticos que o SRE precisa investigar.

Observe que a diferença entre P90 e P99 pode ser de 2x a 10x, dependendo da saúde do sistema. Um servidor saudável terá P90 e P99 relativamente próximos; um servidor degradado terá um P99 desproporcional, indicando que uma pequena fração dos usuários sofre uma experiência drasticamente pior.

Boxplot: Resumo Visual dos Cinco Números

O **Boxplot** (diagrama de caixa) sintetiza em uma única imagem os cinco valores fundamentais de toda distribuição, conhecidos como *five-number summary*:

1. **Mínimo**: O menor valor observado (excluindo outliers).
2. **Q1 (1º Quartil — 25%)**: 25% dos dados estão abaixo deste valor.
3. **Q2 (Mediana — 50%)**: O valor do meio.
4. **Q3 (3º Quartil — 75%)**: 75% dos dados estão abaixo deste valor.
5. **Máximo**: O maior valor observado (excluindo outliers).

Valores que excedem $1.5 \times \text{IQR}$ (Intervalo Interquartil = $Q3 - Q1$) são representados como pontos isolados — os **outliers**.

Para o SRE, o Boxplot é uma ferramenta rápida de diagnóstico. Se a mediana estiver “baixa” na caixa, mas o *whisker* superior for muito longo e houver muitos outliers acima dele, tem-se a confirmação visual de que uma cauda longa de latência existe e precisa ser investigada. Comparar boxplots lado a lado (por exemplo, um para requisições GET e outro para POST) revela instantaneamente se algum método HTTP sofre mais degradação do que outro.

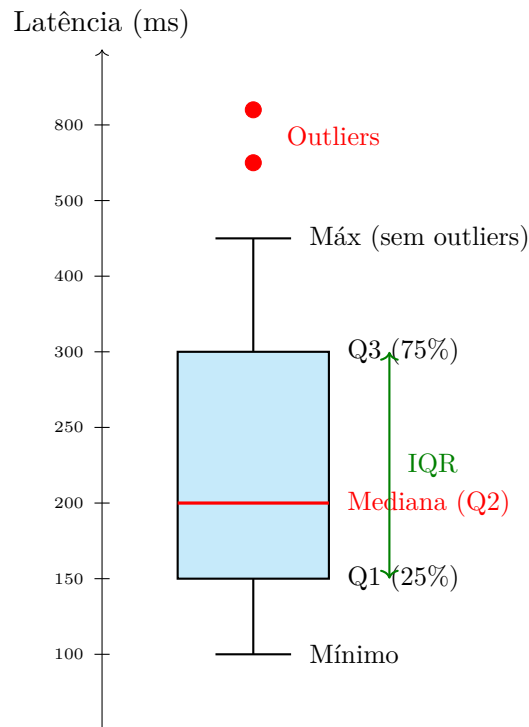


Figura 6.4: Anatomia de um Boxplot: a caixa representa o intervalo interquartil (IQR), a linha vermelha interna é a mediana, e os pontos isolados são outliers.

6.2 Processamento de Dados com Python e Pandas

Para lidar com dezenas de milhares de linhas, ferramentas manuais como planilhas tornam-se ineficientes. A abordagem padrão de engenharia para extração de dados envolve o uso de **Expressões Regulares (Regex)** combinadas com a biblioteca **Pandas** (MCKINNEY, 2022) em Python.

A expressão regular atua como um gabarito que "encaixa" em cada linha do texto e recorta as peças exatas (grupos nomeados). Veja o trecho de código principal utilizado no processamento do estudo de caso:

Listing 6.1: Extração de dados de log via Regex em Python

```

1 import pandas as pd
2 import re
3
4 # Expressao Regex para extrair campos especificos formatados
5 pattern = re.compile(
6     r'(?:(\d+),)?"?'
7     r'(?P<ip>\S+) \S+ \S+ '
8     r'\[(?P<time>[^\]]+)\]'
9     r'"?"?(?P<method>\S+) (?P<url>\S+) (?P<protocol>[^\"]+)"'
10    r'(?P<status>\d{3}) '

```

```

11     r'(?P<size>\d+) '
12     r'""(?P<referer>[~"]*)"" '
13     r'""(?P<agent>[~"]*)"" '
14 )
15
16 data = []
17 with open("access.log", "r", encoding="utf-8") as f:
18     for line in f:
19         match = pattern.search(line)
20         if match:
21             # Converte a linha de texto em um dicionario Python
22             row = match.groupdict()
23             data.append(row)
24
25 # Conversao da lista de dicionarios em uma Tabela analitica
26 df = pd.DataFrame(data)
27
28 # Tratamento de Tipos de Dados Fortes
29 df["status"] = df["status"].astype(int)
30 df["size"] = df["size"].astype(int)
31 df["time"] = pd.to_datetime(df["time"], format="%d/%b/%Y:%H:%M:%S
    ↪ %z")

```

Uma vez organizados no DataFrame ('df'), ganha-se a capacidade de realizar agregações matemáticas, filtros avançados e geração gráfica quase instantaneamente.

6.3 Análise Estrutural e Descritiva

Um dos primeiros passos da análise é rodar comandos descritivos gerais, como o 'df.describe()', para obter o perfil estatístico básico do lote de 10.001 registros extraídos do arquivo contendo latência:

A leitura inicial dessa estrutura revela que a média do tamanho das respostas é de ≈ 12.6 KB, mas a presença de um valor máximo extremo de mais de 565 KB sugere uma cauda longa na distribuição de dados, indicando provavelmente arquivos esporádicos pesados (como imagens de alta resolução). O mesmo fenômeno é visível na latência.

Tabela 6.1: Resumo Descritivo (Padrão Pandas)

	status	size (bytes)	latency_ms
contagem	10001	10001	10001
média	210.49	12635.96	261.43
desvio (std)	39.88	29465.99	251.15
mínimo	200.00	0.00	70.31
25%	200.00	1990.00	162.90
50% (mediana)	200.00	4120.00	191.63
75%	200.00	11617.00	263.43
máximo	502.00	565547.00	4857.70

6.4 Análise de Latência: A Importância do Percentil 90

Quando discutimos o tempo de resposta em arquiteturas Web, a média (*mean*) é uma métrica traiçoeira. No nosso caso, a média de latência é de 261.43 ms. No entanto, mais da metade dos usuários (a mediana dos 50%) experimenta tempos abaixo de 191.63 ms.

Isso ocorre porque as distribuições de latência costumam ser altamente assimétricas e distorcidas para a direita (com uma cauda longa de requisições extremamente lentas). Um pico máximo espúrio de 4.8 segundos puxa a média brutalmente para cima.

6.4.1 O Foco no Fim da Cauda

Por essa razão, o SRE ignora a média em SLOs formais e prioriza medidas de **Percentil** (como P90, P95 ou P99). Ao gerar os dados pelo Pandas no limite de 90%, obtemos que:

Percentil 90 (P90) de latência: O valor máximo que engloba 90% de todas as requisições mais rápidas do sistema.

Se o P90 for de 400 ms, sabemos com certeza gráfica que 90% dos nossos usuários tiveram sua resposta em menos de 400 ms. O objetivo da engenharia de desempenho é empurrar progressivamente o P90, P99 e o valor máximo para baixo, otimizando o "pior cenário" da experiência do usuário, em vez de tornar o caso médio milissegundos mais rápido.

6.5 Calculando Disponibilidade (Availability) e Error Rate

Uma das premissas de Sommerville e do SRE do Google trata os erros na faixa 4xx como erros provocados pelo cliente e a faixa 5xx (Internal Server Errors, Gateways, etc.) como

falhas reais do servidor subjacente.

No estudo de caso, caso assumamos uma definição conservadora de erro cobrindo códigos ≥ 400 , a análise da incidência seria a base para calcular as obrigações da nossa equipe.

Se, das 10.001 requisições processadas, identificarmos 485 respostas com falhas (sejam elas quais forem de acordo com a regra de negócio da tabela):

$$\text{Total de Erros} = 485$$

$$\text{Total de Requisições} = 10.001$$

As duas fórmulas centrais SRE calculadas nesse espaço de tempo seriam:

$$\text{Error Rate} = \frac{\text{Total de Erros}}{\text{Total de Requisições}} \times 100 = \frac{485}{10001} \times 100 \approx 4,85\%$$

$$\text{Availability} = \frac{\text{Sucessos}}{\text{Total}} \times 100 = \frac{9516}{10001} \times 100 \approx 95,15\%$$

6.5.1 O Teste do SLA e dos Noves

Tendo uma janela de tempo de análise, digamos 1 semana, e descobrindo a disponibilidade efetiva baseada em requisição de 95,15%, voltamos à tabela de SLOs ("noves").

Um acordo de nível de serviço exigindo **99,9% ("Três Noves")** de disponibilidade não foi cumprido sequer de perto por esta janela. Com apenas 95% de sucesso, o *Error Budget* do sistema foi massivamente consumido no período correspondente. Sob políticas rígidas de SRE, as implantações da aplicação seriam paradas e a *feature team* mudaria seu foco de desenvolvimento ágil para a eliminação das falhas-raiz e contenção de defeitos subjacentes (detecção e prevenção explorados nos capítulos iniciais).

6.6 Discussão e Causa Raiz

A beleza de ter os dados unificados no Pandas está na correlação imediata de variáveis. Pelo caderno IPython gerado, nota-se que falhas do tipo estado ≥ 500 estavam intrinsecamente correlacionadas a penalidades simuladas na distribuição Normal (com latências sendo empurradas a +800 milissegundos).

Isso reproduz o comportamento mais fidedigno de um banco de dados relacional falhando sob saturação de disco ou CPU: conexões travam, *threads* entram em fila de espera (inflacionando exponencialmente a latência da ponta) até resultarem em *timeouts* transacionados como ‘502 Bad Gateway’ ou ‘504 Gateway Timeout’.

A análise do referencial superior de latência (P90+) inevitavelmente cruza URLs lentas

com IPs de alto fluxo na malha do sistema, caracterizando fortes indícios de acessos de máquinas automáticas ou de *crawlers* consumindo ineficientemente arquivos ricos demais não-otimizados em *cache*. A solução para otimização do P90 envolveria cache em borda (CDNs), compressão ativa de payload (diminuindo o uso de rede ativamente) e isolamento dos robôs identificados pelos campos de agente do usuário nos logs.

Capítulo 7

Resposta a Emergências

A confiabilidade de um sistema não depende apenas de quão raramente ele falha, mas de quão rápido e eficientemente a equipe consegue restaurá-lo quando as falhas inevitavelmente ocorrem. Neste capítulo, exploraremos os princípios fundamentais, as dinâmicas de treinamento e os estudos de caso relacionados à resposta a emergências, baseando-nos fortemente nas práticas de *Site Reliability Engineering* (SRE) desenvolvidas ao longo de décadas de operação pelo Google.

Ao operar sistemas em larga escala, deve-se partir da premissa de que falhas não são anomalias impossíveis, mas sim eventos estatisticamente garantidos. A diferença entre um serviço de alta qualidade e um amador reside na forma estruturada e calma com que a organização absorve o impacto e retorna ao estado de operação funcional.

7.1 Métricas Fundamentais: MTTF e MTTR

A confiabilidade não é um conceito abstrato; ela pode e deve ser medida através de indicadores de tempo. Ao observar um sistema ao longo de seu ciclo de vida, duas métricas se destacam:

- **MTTF (Mean Time To Failure – Tempo Médio para Falhas):** Avalia a frequência com que os problemas ocorrem. Um sistema com alto MTTF passa bastante tempo operando corretamente antes de apresentar um defeito.
- **MTTR (Mean Time To Repair – Tempo Médio para Correção):** Mede a rapidez com que a equipe de resposta consegue diagnosticar, conter e devolver a saúde ao sistema após uma interrupção.

Do ponto de vista prático e da experiência do usuário, a métrica mais relevante na avaliação da eficiência da resposta a incidentes é o MTTR. Quanto menor for o tempo de recuperação, menor será o impacto percebido pelo cliente.

A figura a seguir ilustra a relação entre essas métricas no horizonte de tempo operacional.

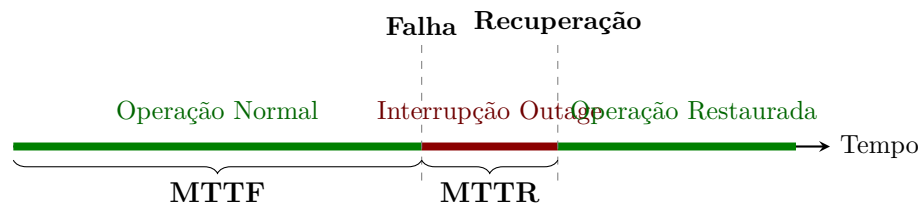


Figura 7.1: Representação visual do ciclo de vida de um incidente, destacando o tempo operando corretamente (MTTF) versus o tempo de reparo (MTTR).

7.2 O Fator Humano: Seres Humanos Acrescentam Latência

Um princípio consagrado pela Engenharia de Confiabilidade moderna é que a intervenção humana durante uma emergência quase sempre acrescenta latência excessiva ao processo de recuperação. Diferente de scripts e *health checks* automatizados, os seres humanos precisam ser acordados (caso o alarme toque de madrugada), precisam ler gráficos, compreender o contexto, criar hipóteses e, só então, agir.

Isso nos leva a uma constatação: um sistema capaz de detectar e evitar (ou mitigar) emergências por meio de intervenções automáticas — como redirecionamento de tráfego, auto-escalonamento ou reinicialização de *Pods* defeituosos — terá uma disponibilidade geral infinitamente superior a um sistema dependente de uma pessoa lendo um manual.

Contudo, quando a automação falha e a intervenção humana se torna inevitável, é crucial pensar nas melhores práticas de mitigação. A resposta analítica e calma não surge espontaneamente sob estresse severo sem uma grande quantidade de preparação e ferramentas adequadas.

7.2.1 O Manual de Regras (Runbook / Playbook)

A principal ferramenta preventiva contra a latência cognitiva e o erro humano é o *Runbook* (frequentemente também chamado de *Playbook*). O cenário ideal de gerenciamento de plantão (*on-call*) exige que o time construa manuais pragmáticos. Um bom runbook permite que qualquer engenheiro da equipe resolva o problema com eficiência, mesmo que esteja fadigado ou tenha pouca familiaridade com os meandros mais profundos daquele microserviço específico.

Os melhores runbooks não se limitam a listar comandos para "copiar e colar". Eles orientam sobre *como pensar* sobre o estado do sistema. Normalmente contêm:

- **Sintomas conhecidos:** O que esse alarme específico significa exatamente?
- **Ações de Mitigação Imediata:** Como conter o "sangramento" antes de procurar a causa real. (ex: "Desligue temporariamente o servidor de cache secundário").
- **Arquiteturas simplificadas:** Diagramas essenciais ajudando a visualizar dependências.
- **Checklists e Painéis (Dashboards):** Onde analisar rapidamente os logs do erro.

7.3 Práticas de Preparação: A Cultura SRE

O Google adotou dinâmicas corporativas ativas para manter suas equipes perfeitamente afinadas sob alta pressão, de forma a não depender da "sorte". Uma das metodologias mais famosas é o *Wheel of Misfortune* (Roda do Infortúnio), uma simulação recorrente de incidentes semelhante a um jogo de *RPG* de mesa aplicado ao contexto de engenharia de software.

7.3.1 A Dinâmica do “Jogo”

A dinâmica é estruturada para imitar a urgência de uma crise real, forçando os engenheiros a tomarem decisões operacionais e de comunicação. Consiste em duas partes principais:

- **O Mestre (Game Master):** Um engenheiro sênior que conhece profundamente o sistema escolhido. Ele baseia sua aventura num incidente verdadeiro (normalmente retirado do repositório de *Postmortems* do passado). O mestre descreve cenários como: *“São 3h da manhã do seu turno de plantão. Seu Pager toca. O alerta diz ‘Latência de 500ms no serviço de Checkout’. O que você faz?”*.
- **O Jogador (Em treinamento):** Geralmente um engenheiro júnior ou recém-alocado na escala de plantão. Ele deve demonstrar onde clicaria, que gráficos analisaria e quem acionaria.

Essa prática possui duas grandes vantagens para as rotinas corporativas:

- **Validação Contínua dos Manuais (Playbooks):** Se durante a simulação o jogador afirmar *“Vou consultar a seção 4 do runbook”*, e as instruções ali contidas estiverem desatualizadas, a equipe descobre isso em “tempos de paz”. O documento é imediatamente corrigido pós-jogo, antes de custar dinheiro real em um incidente verdadeiro.

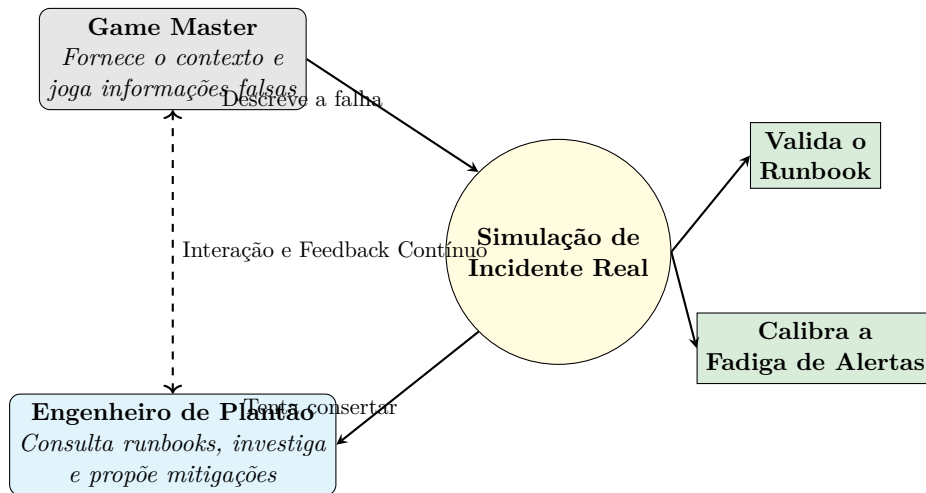


Figura 7.2: Fluxo esquemático da dinâmica *Wheel of Misfortune*. A simulação alimenta melhorias diretas na qualidade dos guias operacionais da equipe.

- **Educação contra a Fadiga de Alerta:** O mestre insere *insights* confusos e alertas secundários irrelevantes no fluxo, forçando a equipe a exercitar o isolamento do sinal em meio ao ruído de fundo. Isso calibra o perfil técnico para lidar com sistemas excessivamente tagarelas.

7.4 As Coisas Quebram: A Vida É Assim

Como discutido, as coisas invariavelmente operam de modo falho na vida real. Uma resposta ágil e apropriada a uma emergência não é um dom natural que emana da personalidade de super-heróis da TI. Pelo contrário: exige vasta preparação, treinamento metódico e suporte financeiro/gerencial.

As empresas mais saudáveis e resilientes do mercado são caracterizadas por criar um ambiente onde é permitido que as equipes gastem tempo, energia e às vezes até percentuais controlados do seu orçamento de erros (*Error Budgets* e *uptime*) testando seus limites em produção. O verdadeiro valor está na construção de um **processo de aprendizado institucional**.

7.4.1 O Que Fazer Quando os Sistemas Falham

Em SRE, criou-se um consenso prático do que a equipe de frente deve fazer ao bater de frente com o "incêndio". A primeira e indiscutível regra é **Não entre em pânico**. Se você, na cadeira de comando, sente a sobrecarga cognitiva do pânico paralisá-lo, é vital focar exclusivamente em um único vetor: atenuar o impacto sobre o usuário do lado de fora antes de focar na elegância da correção.

Dessa forma, os passos elementares sugeridos para condução firme são:

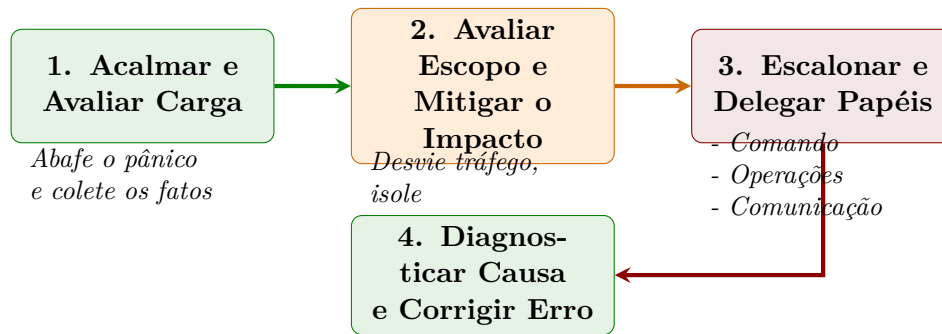


Figura 7.3: Passo a passo inicial para gerenciar um incidente sem se deixar levar pelo pânico. Foco na recuperação imediata do tráfego através do bypass antes da caça aos bugs intrincados.

1. **Não entre em pânico.** Respire fundo. O pânico turva o raciocínio clínico.
2. **Avalie a situação:** Qual o escopo real e o impacto da falha? Descobrir a abrangência é vital. Muitas vezes curtos tempos de inatividade integral (como desligar uma funcionalidade inteira para reiniciá-la) costumam ser muito menos destrutivos do que longos períodos crônicos de degradação intermitente.
3. **Escale o problema:** Não seja o herói solitário. Ao deduzir que o tempo de resolução se estenderá ou ultrapassará a sua faixa de conhecimento técnico, envie um chamado a outras equipes imediatamente. Durante um incidente grave, divide-se a gestão em sub-funções (O Comandante do Incidente ditando o ritmo, Operadores escrevendo os códigos de correção e os Comunicadores publicando as atualizações para as partes interessadas de alto escalão).

7.5 Estudos de Caso SRE: Taxonomia das Emergências

Nem todo incidente surge misteriosamente durante a madrugada impulsionado por um ataque ou falha de hardware exterior. Muitas vezes, as emergências mais custosas nascem da própria ação da engenharia da empresa sob condições não perfeitamente previsíveis. Avaliaremos três categorias cruciais observadas na história de respostas do Google SRE.

7.5.1 1. Emergência Induzida por Testes (O Caso do DB MySQL)

Um caso histórico exemplifica os testes proativos se tornando emergências ativas. O Google mantém processos conhecidos como *DiRT* (*Disaster Recovery Testing*), voltados inteiramente para agendar sabotagens controladas na própria rede e observar como a resiliência projetada reage. Em um plano específico, os engenheiros quiseram provar que

a queda bruta de **um** único banco de dados de teste (dentre milhares de réplicas de MySQL) seria completamente suprimida pelo roteamento natural.

O objetivo era bloquear o acesso desse pequeno nó para descobrir dependências ocultas espúrias. Quando ativado, porém, o sistema divergiu violentamente de todas as premissas: a sub-rede colapsou dependências assíncronas invisíveis transversais e desencadeou velozmente uma falha generalizada nos serviços internos da camada de negócio e no acesso externo à plataforma principal.

O Aprendizado: Foi um sucesso incontestável em encontrar vulnerabilidades ocultas, apesar da dor profunda a curto prazo impingida. O abandono imediato do teste e a transição instantânea da equipe SRE do papel de "Testadores" para "Comandantes de Incidente" atenuaram a crise velozmente (devido ao alerta iminente). Falhas controladas proativas destrincham as arquiteturas cegas, que, deixadas sozinhas, teriam explodido espontaneamente no futuro sob condições incontroláveis num pico de mercado desastroso.

7.5.2 2. Emergência Induzida por Alterações de Configuração

Alterações de infraestrutura e atualizações de rotina são poços de perigos constantes. Em uma data que parecia ser comum (uma sexta-feira), decidiu-se implantar globalmente uma mudança estrita em arquivos de configuração da proteção principal contra tráfego anômalo. Sendo percebida falsamente como de risco insignificante, uma varredura cautelosa (*Canary Deployment*) rigorosa não foi aplicada com as devidas pausas.

Ao apertar o gatilho da difusão de arquivos, algumas combinações exóticas de palavras-chave subadjacentes não foram cobertas por testes automatizados, servindo de estopim para contínuos *crashes* catastróficos por debaixo dos panos de todos os sistemas de resposta voltadas à internet pública. Pior ainda: a comunicação interna de rádio e mensagens cruciais das ferramentas das equipes de plantão dependia da própria matriz roteadora afetada pelo *crash*. Formou-se uma arquitetura hostil de **recursividade ou dependência circular**.

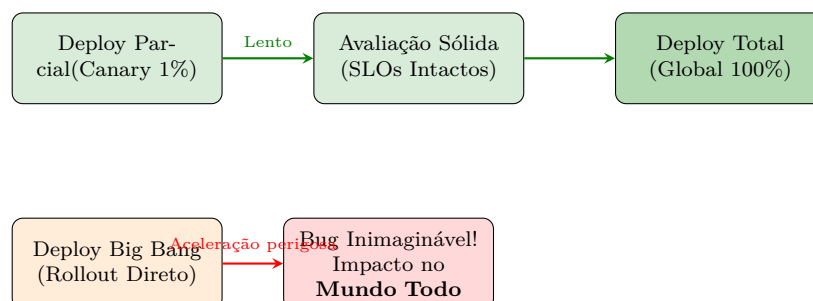


Figura 7.4: Comparativo Metodológico: um fluxo de *Canary Deployment* inteligente restringe as sequelas localmente num subconjunto de máquinas antes da aprovação ampla, evitando que alterações imaturas incendeiem a empresa inteira formidavelmente ("Deploy Big Bang").

O Aprendizado: Fundamentalmente, nunca existe alteração de "zero risco" em ambientes fortemente amarrados. Avaliações graduais e imersivas (*Canary*) poderiam ter contido o desastre no minuto um em um raio minúsculo. Em adição a isso, atestou-se a obrigação arquitetural em extirpar as famigeradas dependências circulares SRE: *A sua ferramenta de comunicação contra incidentes de rede interna não pode, paradoxalmente, depender da malha de sua própria rede no instante em que for requerida para ressuscitá-la.*

7.5.3 3. Emergência Induzida por Processos Automáticos (Caso Diskerase)

Na incessante necessidade de otimização de tempo horizontal, o Google escreve automações sistêmicas robóticas avassaladoras. Durante a desativação burocrática massiva de máquinas sucateadas antigas, duas requisições consecutivas paralelas foram ordenadas ao servidor robótico despachante, criadas puramente por dupla submissão negligente de um operador num web console (F5 duplo).

Ao processar velozmente a primeira solicitação completa e tentar atuar sobre o segundo pacote *input* processado por descompassos de latência, a API da máquina remetente devolveu uma resposta de localização vazia. Um pormenor ou *glitch* na programação original da leitura do sistema levou a inteligência de script a concluir perigosamente: *"Ah, a variável do agrupamento encontra-se vazia, sem filtros textuais estipulados! Logo a string vazia equivale a um metacaractere wildcard: 'selecione tudo'".* O robô iniciou de prontidão implacável o fluxo letal conhecido na *Big-Tech* como *Diskerase* (A formatação e reescrita de discos definitivos magnéticos) sobre os racks alvos — enviando máquinas em data centers pelo mundo inteiro (ainda servindo tráfego produtivo vital!) para o formato terminal sumário inclemente, obliterando tráfego funcional instantaneamente em ritmo avassalador. Apenas restou a intervenção de desligamento manual do núcleo de *scripts* para refreá-lo ("arrancar a tomada").

O Aprendizado: Na engenharia de infraestrutura massiva automatizada, parâmetros vazios frequentemente destoam do seu sentido linguístico de segurança inerte e convertem-se facilmente ao nefasto sinônimo literal de *"Tudo de forma indiscriminada"*. Conclui-se que não se deve confiar no discernimento semântico de processadores em massa de APIs para injetar lógicas de destruição irreversíveis baseados puramente em condicionais. Portanto, operações de extinção imperam inegociavelmente da presença de *Sanity Checks* extremas (Verificações de Sanidade): baluartes bloqueadores duros que barram estatisticamente a tarefa se o limite numérico exceder a expectativa tática regular (ex. *"Você está pedindo para deletar 5.000 instâncias num cluster quando a média diária é 4? Recusado por defeito!"*) obrigando assim exigências de bypass rigorosas baseadas em tokens operacionais humanos.

7.6 Aprenda com o Passado: O Valor da Cultura *Blameless*

A base robusta da engenharia de confiabilidade perante acidentes desastrosos não se consagra tentando reprimir ou ocultar relatórios corporativos com vergonha do tempo de inatividade, mas em abraçá-los minuciosamente catalogando-os na biblioteca digital indissolúvel pelo emprego majestoso dos chamados **Postmortems** (Parecer Pós-Morte dos Incidentes).

Esta dinâmica foge da retórica corporativista tradicional voltada a demissões iradas sumárias ou caça aos programadores expiatórios isolados; materializa-se na confecção minuciosa científica de atestados pragmáticos absolutamente *isentos de julgamento passional individual*. A cultura do Google exalta abertamente a prerrogativa suprema sagrada do postmortem "**sem culpa**" (*Blameless Postmortems*). Admite-se com tranquilidade o paradigma natural de que toda falta individual ou clique trágico é a ponta frágil da materialização de processos permissivos mal arquitetados, furos em esteiras de testes deficientes, validações fracas na interface visual administrativa (*UI*) ou carência generalizada de barreiras protetivas elementares (*guardrails*), descartando a ilusão que o "*erro humano não intencional e cansado*" é o causador da inoperabilidade escalável. Demitir o maquinista é infinitamente menos proveitoso na evitação futura do acidente do que reforçar a geometria da ponte metálica.

A construção exigirá relatórios objetivos pragmáticos explicitando friamente e de trás para a frente a pergunta estrutural: *Que eventos exatos encadeados cronologicamente propiciaram este cataclisma em milissegundos? Quais recursos preventivos definitivos em engenharia podem dismantelar logicamente a reedição dos vetores combinatórios neste caldeirão em colapsos similares?*

7.6.1 Testes Proativos e o Círculo Virtuoso da Resiliência Sistêmica

Modernos aglomerados sistêmicos interconectados via microsserviços carecem da intrepidez filosófica prática contínua embasada e martelada nas frentes indagadoras provocativas reflexivas: "*E se...?*"

"E se a malha matriz do cluster de roteamento perdesse subitamente integridade ou energia neste momento de pico bancário diário, os scripts adormecidos do repositório secundário responderiam ao failover tempestivamente nos prometidos parques cinco segundos projetados pelas planilhas estatísticas originais?"

Incentivar simulações incertas controladas corrói o otimismo nocivo baseado em presunções irreais hipotéticas não atestadas e retestadas sob estresse verdadeiro e fogo analítico da realidade imperativa, evitando falsas seguranças letárgicas herdadas da complexidade abstrata e nebulosa adormecida não tangível. As redes caóticas e imprevisíveis exigem testes proativos frequentes sem qualquer receio ou concessão à estabilidade mentirosa silenciosa de momentos mortos temporários.

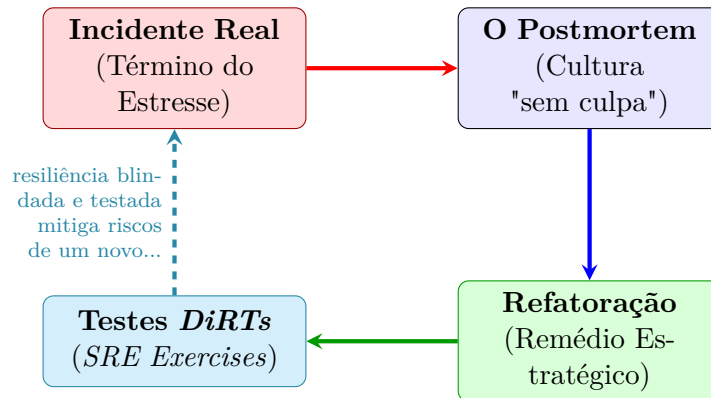


Figura 7.5: O Círculo Virtuoso e Orgânico da Melhoria Contínua. Nenhum impacto degradante sistêmico deve ser superado e meramente arquivado sem nutrir efetivamente as contramedidas ativas estruturais retestadas rigorosamente nas etapas sucessivas de ensaios futuros simulados preventivos.

Por conseguinte, debaixo deste amadurecimento tático impiedosamente moldado nos picos intensos caóticos inerentes não extirpáveis à engenharia confiável resolutive — não se almeja passivamente ou ilusoriamente afastar o colapso pontual pela presunção ingênua e inatingível que as arquiteturas do processamento computacional são estanques divinas repletas de absoluta e inabalável perfeição infalível. Visa-se sim enfrentar e combater implacavelmente os vetores de falha com métodos rigorosamente afiados e preestabelecidos com orquestrações resolutivas infalíveis e respostas ágeis e táticas imperturbáveis mitigadoras. A atuação sistêmica sólida e flexível alicerçada na cultura resoluta interposta frente ao infortúnio alinha-se cirurgicamente na simbiose irreduzível ao intelecto do capital humano treinado lapidado e fortalecido incisivamente por esse embate contínuo experimental constante orgânico irreduzível sob as simulações do rigor implacável dos ensaios em crises controladas. A roda contínua da resiliência jamais detém seu curso nas metodologias da vitoriosa resiliência de produção em alta escala confiável contemporânea escalável proeminente.

Capítulo 8

Engenharia de Segurança

Historicamente, a computação se preocupava majoritariamente com a corretude lógica e o desempenho. No entanto, à medida que o software passou a controlar processos físicos críticos — de aeronaves e usinas nucleares a dispositivos médicos implantados no corpo humano — uma nova dimensão de qualidade tornou-se imperativa: a **segurança** (*safety*).

Diferente da confiabilidade, que foca na probabilidade de o sistema operar sem falhas por um período de tempo, a engenharia de segurança foca em garantir que o sistema não cause danos catastróficos, como morte, lesão ou prejuízo ambiental grave, mesmo diante de comportamentos imprevistos. Neste capítulo, exploraremos os fundamentos desta disciplina, as técnicas de análise de perigos e as estratégias de projeto para sistemas onde a falha não é uma opção.

8.1 O Software como Componente Crítico: O Caso de Varsóvia

Para compreendermos a importância da engenharia de segurança, é vital analisar incidentes reais onde a tecnologia falhou em proteger vidas. Um dos casos mais emblemáticos ocorreu em **14 de setembro de 1993**, no Aeroporto de Varsóvia, envolvendo um Airbus A320 da Lufthansa.

Durante o pouso sob condições de chuva intensa e ventos cruzados, a aeronave enfrentou um fenômeno conhecido como *aquaplanagem*. O sistema de software de controle da frenagem, projetado para ser "inteligente", possuía requisitos de segurança específicos para evitar que os reversores de empuxo e os freios de solo fossem acionados enquanto o avião ainda estivesse em voo. Para o software, o avião só era considerado "em solo" se duas condições fossem atendidas:

1. Os sensores de pressão nos trens de pouso principais detectassem um peso superior a 12 toneladas.

2. As rodas estivessem girando em uma velocidade específica.

No dia do acidente, devido à fina camada de água na pista, o avião aquaplanou. As rodas não giraram na velocidade esperada e a pressão nos sensores flutuou. Como resultado, o software "concluiu" que a aeronave ainda estava em voo e **bloqueou** o acionamento dos sistemas de frenagem total por nove segundos cruciais. A aeronave ultrapassou o final da pista e colidiu com um aterro, resultando em duas mortes e 54 feridos.

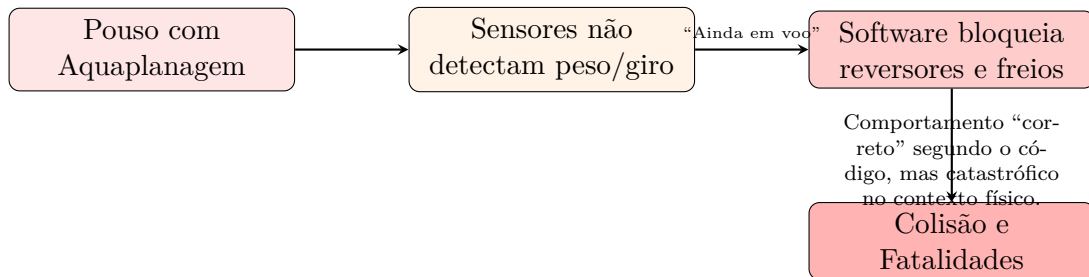


Figura 8.1: Fluxo do acidente em Varsóvia: a lógica de segurança do software tornou-se o próprio vetor do perigo.

Este caso ilustra uma lição fundamental: o software funcionou exatamente como foi especificado. Não houve um "bug" de codificação no sentido tradicional, mas sim uma **falha de especificação de segurança**. Os engenheiros não previram que situações físicas perfeitamente possíveis (aquaplanagem) invalidariam os critérios de detecção de solo.

8.2 O que é Segurança (*Safety*)?

Em engenharia de sistemas, o termo **segurança** (frequentemente referido pelo termo em inglês *safety*) possui uma definição muito restrita e distinta da segurança da informação (*security*).

Segurança (*Safety*): É a propriedade de um sistema de operar, sob condições definidas, sem causar falhas catastróficas que resultem em morte, lesão pessoal ou dano ambiental em larga escala.

É importante destacar que um sistema que falha frequentemente (baixa confiabilidade) pode ainda ser considerado seguro se todas as suas falhas forem "seguras" (ex: um trem que para automaticamente se houver perda de energia). Por outro lado, um sistema que nunca para de funcionar (alta disponibilidade) pode ser extremamente perigoso se continuar operando incorretamente em uma situação de risco.

8.2.1 *Safety* versus *Security*

No Brasil, usamos a palavra “segurança” para ambos os conceitos, o que gera confusão. Para fins técnicos, devemos distinguir:

- **Safety:** Proteção do **mundo** contra o **sistema**. Foca em falhas acidentais e erros de projeto que causam danos físicos.
- **Security:** Proteção do **sistema** contra o **mundo**. Foca em ataques deliberados, acessos não autorizados e integridade dos dados.

Ambos são pilares da **Dependabilidade**, conforme discutimos no capítulo anterior, mas exigem técnicas de engenharia radicalmente diferentes.

8.3 A Segurança na Árvore da Dependabilidade

A segurança é um dos atributos primordiais da dependabilidade de um sistema. Conforme ilustra o diagrama abaixo, ela compartilha espaço com propriedades como confiabilidade e disponibilidade, mas possui uma métrica de sucesso diferente: a ausência de acidentes.

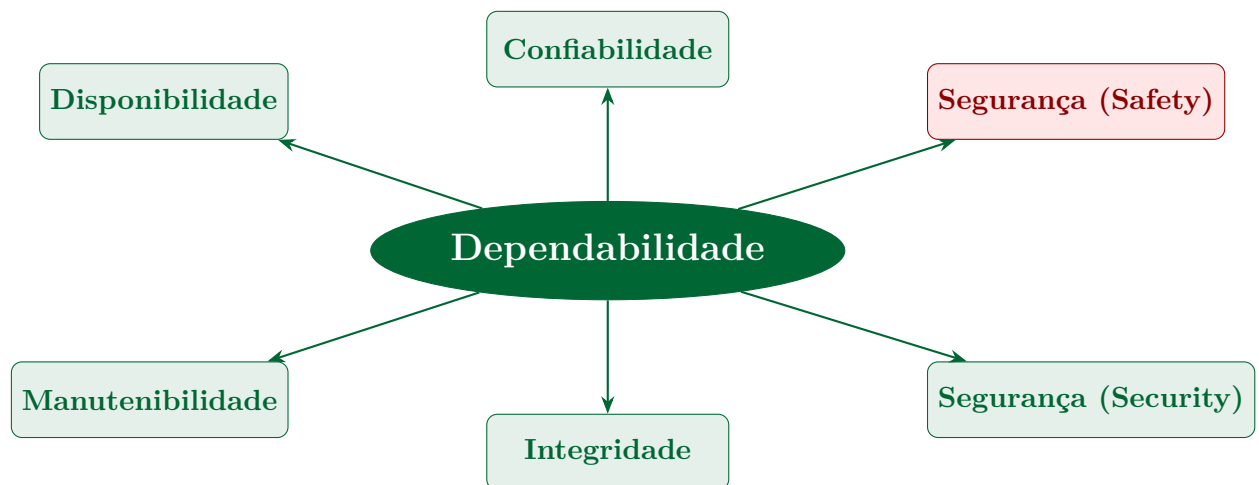


Figura 8.2: Taxonomia da Dependabilidade: A Segurança (*Safety*) como pilar crítico focado na prevenção de catástrofes físicas.

8.4 O Duplo Papel do Software na Segurança

Em sistemas modernos, o software não é apenas um componente; ele atua como o "cérebro" e o "vigia" do sistema físico. Podemos classificar sua atuação em dois papéis fundamentais:

8.4.1 Papel 1: Controle Direto

Neste papel, o software é responsável por tomar decisões e comandar atuadores físicos. Exemplos incluem o controle de injeção de combustível, o ajuste de superfícies de comando de um avião ou a dosagem de radiação em uma máquina de radioterapia. Uma falha aqui se traduz diretamente em uma ação física perigosa.

8.4.2 Papel 2: Monitoramento e Salvaguarda

O software atua como um sistema de proteção independente. Ele monitora sensores, verifica se o sistema físico está operando dentro de limites seguros e, se detectar uma anomalia, toma medidas de mitigação (como ativar um desligamento de emergência). Ironicamente, a falha deste software de monitoramento pode não causar um acidente imediato, mas deixa o sistema "cego" e indefeso contra falhas posteriores nos componentes de hardware.

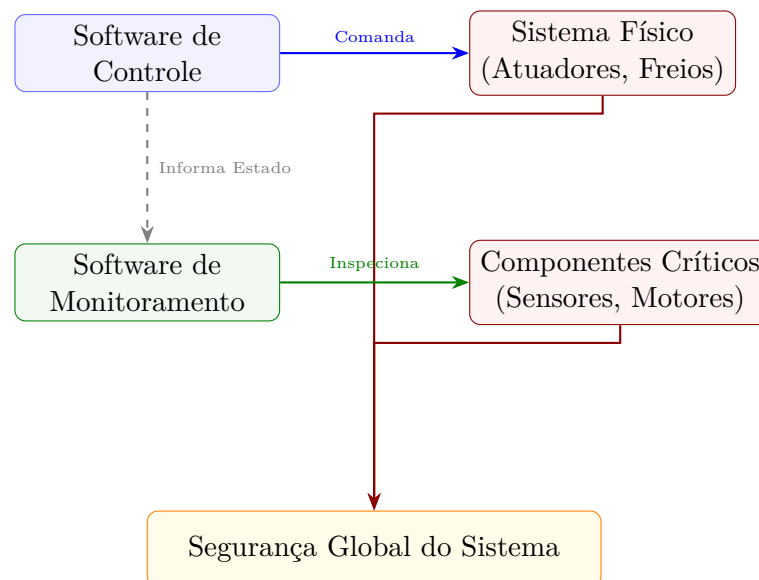


Figura 8.3: Interação entre softwares de controle e monitoramento na composição da segurança operacional.

8.5 A Falácia: Se é Confiável, é Seguro

Um dos erros mais comuns e perigosos na gestão de projetos é acreditar que o aumento da confiabilidade levará automaticamente ao aumento da segurança. Como vimos no caso de Varsóvia, um sistema pode ser 100% confiável (executar seu código sem nenhuma falha de interrupção ou erro de lógica interna) e ainda assim ser inseguro (o código em si comanda uma ação que leva ao acidente).

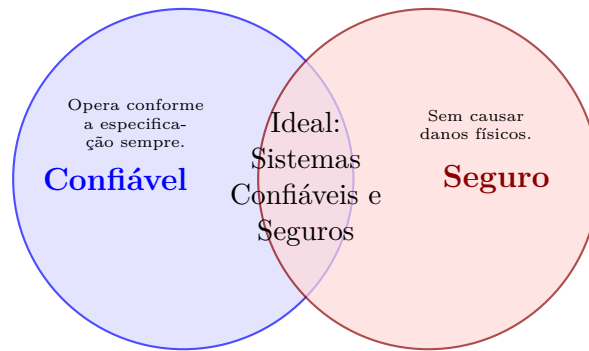


Figura 8.4: Relação entre Confiabilidade e Segurança: Propriedades que se interceptam, mas não se sobrepõem totalmente.

Existem quatro razões fundamentais pelas quais sistemas tecnicamente confiáveis podem falhar em segurança:

1. **Defeitos Inativos (Bugs Latentes):** Um software pode operar perfeitamente por anos em um regime normal até que uma combinação rara de estados do sistema (ex: bissexto + horário de verão + falha de rede) ative um caminho de execução nunca antes testado.
2. **Especificações Incompletas:** É impossível para um ser humano prever todas as interações do software com o ambiente físico. Se a especificação não diz o que o software deve fazer em uma situação de emergência, ele terá um comportamento indefinido.
3. **Mau Funcionamento de Hardware:** O software assume que os sinais recebidos dos sensores são válidos. Se um sensor falha e começa a enviar dados ruidosos ou "congelados", o software pode tomar decisões catastróficas baseado em mentiras eletromecânicas.
4. **Erros de Operação:** O software pode executar perfeitamente um comando humano que é válido individualmente, mas letal no contexto. Como discutiremos no exemplo da bomba de insulina, aceitar um comando de "bolus" máximo em um momento de glicemia já baixa é uma falha de segurança, mesmo que o código de entrega da insulina funcione sem erros.

8.6 Análise de Perigos: Antecipando o Desastre

A engenharia de segurança não espera o sistema estar pronto para testar sua segurança; ela utiliza técnicas proativas de **Análise de Perigos** (*Hazard Analysis*) desde as fases iniciais de requisitos.

8.6.1 Árvore de Falhas (*Fault Tree Analysis* — FTA)

A FTA é uma técnica dedutiva (*top-down*) que começa com um "Evento de Topo" indesejado (ex: Acidente) e retrocede logicamente para identificar todas as combinações de falhas de componentes e erros operacionais que poderiam causá-lo. Ela utiliza portas lógicas (AND/OR) para modelar as dependências.

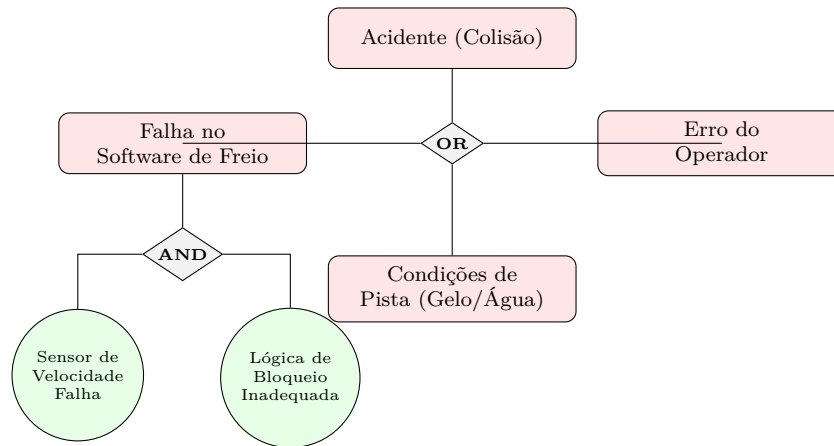


Figura 8.5: Exemplo de Árvore de Falhas: Desconstruindo logicamente as causas de um acidente.

Ao construir uma árvore de falhas, os engenheiros conseguem identificar os chamados **conjuntos de corte** (*cut-sets*): as menores combinações de falhas que levam ao evento de topo. O objetivo do projeto de segurança é garantir que nenhum evento catastrófico dependa de uma única falha (*Single Point of Failure*).

8.6.2 FMEA: Análise de Modos de Falha e Efeitos

Enquanto a FTA olha para baixo a partir do acidente, a **FMEA** (*Failure Mode and Effects Analysis*) olha para cima a partir dos componentes. É uma técnica indutiva (*bottom-up*) onde cada componente do sistema é analisado individualmente: 1. Como ele pode falhar? (Modos de Falha) 2. O que acontece se ele falhar? (Efeitos) 3. Quão grave é esse efeito? (Severidade)

Essa análise resulta em uma matriz de risco que prioriza quais componentes exigem redundância ou proteções adicionais.

8.7 Categorias de Criticidade em Segurança

Nem toda falha tem o mesmo peso. A engenharia de segurança classifica os sistemas e seus riscos em categorias de severidade, o que orienta o nível de rigor exigido no processo de desenvolvimento e certificação.

1. **Catastrófico:** Falhas que resultam em múltiplas mortes ou destruição total de um ecossistema (ex: queda de avião comercial, explosão de reator).
2. **Crítico:** Falhas que causam lesões graves ou mortes isoladas, ou danos ambientais significativos mas contidos.
3. **Marginal:** Falhas que resultam em lesões leves, doenças ocupacionais ou danos contidos ao patrimônio e ambiente.
4. **Negligenciável:** Falhas que não resultam em lesões e têm impacto ambiental mínimo, embora possam causar atrasos operacionais.

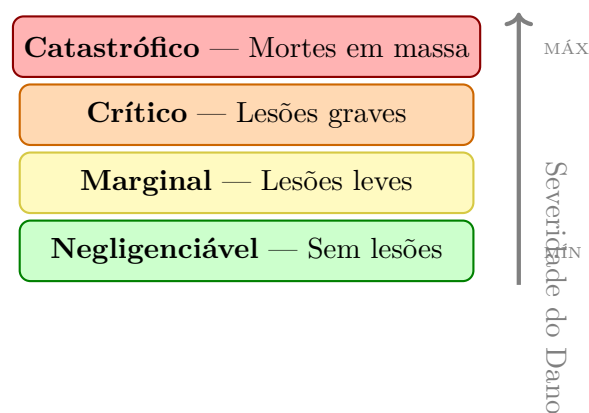


Figura 8.6: Escala de Severidade: A classificação do risco determina o investimento em mecanismos de proteção.

8.8 Requisitos de Segurança: O Poder do “Não Deve”

Diferente dos requisitos funcionais tradicionais, que descrevem o que o sistema **deve fazer** para gerar valor, os requisitos de segurança frequentemente focam no que o sistema **não deve fazer** para evitar desastres. Esses são conhecidos como requisitos de exclusividade ou restrição (*Shall Not Requirements*).

Exemplos de Requisitos “Não Deve”

- **Aviação:** “O sistema **não deve** permitir o acionamento do empuxo reverso se as rodas não estiverem em solo.” (Lição de Varsóvia).
- **Médico:** “A bomba de insulina **não deve** administrar uma dose superior a 10 unidades em um intervalo de uma hora.”
- **Ferroviário:** “O sistema de sinalização **não deve** permitir que dois trens ocupem o mesmo bloco de via simultaneamente.”

O desafio técnico desses requisitos é que eles são difíceis de testar exaustivamente. É muito mais fácil provar que uma funcionalidade existe do que provar que uma combinação de eventos **nunca** ocorrerá. Por isso, a engenharia de segurança recorre a métodos formais e provas matemáticas de propriedades de exclusão.

8.9 Arquitetura e Programação para Sistemas Seguros

Para garantir que o software se comporte de maneira segura mesmo sob estresse ou falha parcial, utilizamos duas estratégias complementares: **Prevenção de Falhas** e **Tolerância a Falhas**.

8.9.1 Prevenção de Falhas (Fuga de Erros)

O foco é impedir que erros humanos de programação cheguem ao sistema final. Isso inclui:

- **Linguagens Fortemente Tipadas:** Uso de linguagens que barram erros de memória e tipos em tempo de compilação (ex: Ada, Rust ou subconjuntos seguros de C++ como o MISRA C++).
- **Proibição de Construções Perigosas:** Evitar o uso de ponteiros soltos, alocação dinâmica de memória em tempo de execução e recursão infinita.
- **Verificação Formal:** Uso de ferramentas matemáticas para provar que uma determinada condição lógica de segurança nunca será violada pelo código.

8.9.2 Tolerância a Falhas (Redundância e Votação)

Assume-se que erros ocorrerão. O sistema deve ser capaz de detectá-los e continuar operando de forma segura. A técnica mais comum é o **Triple Modular Redundancy (TMR)**, ou Redundância Modular Tripla.

Neste modelo, três cópias independentes do software (muitas vezes escritas por equipes diferentes seguindo a mesma especificação — Programação N-versões) processam os mesmos dados. Um componente chamado **Votador** compara os resultados. Se dois canais concordarem e um divergir, o sistema ignora a falha e utiliza o resultado majoritário.

8.9.3 O Estado Seguro (*Safe State*)

Quando o sistema não consegue mais garantir a operação redundante (ex: dois dos três canais falharam), ele deve transitar para um **Estado Seguro**. O estado seguro é uma con-

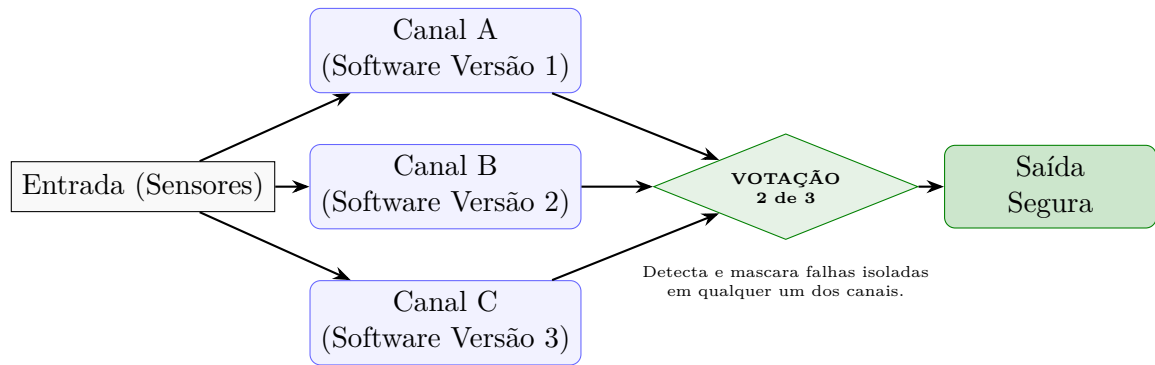


Figura 8.7: Arquitetura TMR: Resiliência através da redundância e diversidade de código.

figuração do sistema onde nenhum dano catastrófico pode ocorrer, mesmo que o sistema pare de prestar seu serviço principal.

- No caso de um semáforo inteligente, o estado seguro pode ser “todos os sinais em vermelho intermitente”.
- Em um processo químico explosivo, pode ser a abertura de todas as válvulas de alívio de pressão.
- No software de um trem, o estado seguro é quase sempre o acionamento dos freios de emergência (parada total).

8.10 Software Crítico Primário vs. Secundário

Nem todo software crítico tem o mesmo nível de proximidade com o perigo físico. Dividimos o software em duas grandes categorias:

1. **Software Crítico em Segurança Primário:** É o software embarcado que controla diretamente o hardware do sistema. Uma falha aqui resulta em um perigo imediato (ex: software da bomba de insulina que injeta dose letal).
2. **Software Crítico em Segurança Secundário:** É o software que gera informações ou modelos que são usados para tomar decisões de segurança, ou que controla processos que podem levar indiretamente a um perigo. Exemplos:
 - Um sistema de CAD usado para projetar o motor de um avião. Se houver um bug no cálculo de fadiga do material, o motor pode falhar fisicamente meses depois.
 - Um sistema de prontuário eletrônico (como o Mentcare) que, se apresentar dados errados, pode levar um médico a prescrever uma dosagem perigosa de medicamento.

Essa distinção é importante pois softwares secundários muitas vezes operam em sistemas operacionais comuns (Windows/Linux) e usam bancos de dados padrão, exigindo uma análise de segurança voltada à integridade dos dados, enquanto softwares primários exigem sistemas de tempo real (*RTOS*) e controle determinístico.

8.11 Especificação Dirigida por Perigos

Diferente de sistemas convencionais, onde os requisitos são guiados por funcionalidades desejadas, sistemas críticos são guiados pela necessidade de evitar **perigos** (*hazards*). O processo é cíclico e foca na redução contínua do risco associado a cada perigo identificado.

8.11.1 Terminologia Fundamental

Para uma análise rigorosa, é preciso utilizar os termos corretos:

- **Acidente:** Um evento não planejado que resulta em dano real (lesão, morte, destruição).
- **Dano:** A medida da perda (ex: morte de um paciente).
- **Perigo (*Hazard*):** Um estado do sistema que tem **potencial** para causar um acidente (ex: porta do elevador aberta enquanto ele se move).
- **Risco:** A combinação da probabilidade de ocorrência do perigo com a gravidade do dano resultante.

O objetivo principal da engenharia de segurança é reduzir o risco a um nível **ALARP** (*As Low As Reasonably Practicable* — Tão baixo quanto razoavelmente praticável), equilibrando custo, funcionalidade e benefício social.

8.11.2 O Fluxo da Especificação

A especificação de segurança segue quatro etapas fundamentais:

1. **Identificação do Perigo:** Brainstorming com especialistas de domínio para listar tudo o que pode dar errado.
2. **Avaliação do Perigo:** Determinar a probabilidade e a severidade.
3. **Análise do Perigo:** Identificar as causas raízes (usando FTA ou FMEA).
4. **Redução do Risco:** Definir requisitos de segurança para prevenir ou mitigar o perigo.

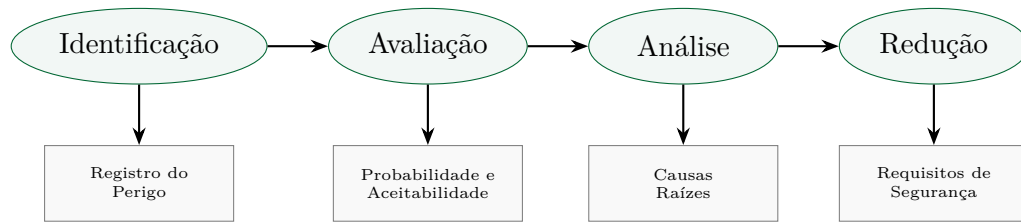


Figura 8.8: Processo iterativo de especificação de segurança guiada por perigos.

8.12 Abordagens Práticas para Mitigação de Riscos

Existem três linhas de defesa que um engenheiro de software pode implementar para lidar com os perigos identificados:

1. **Prevenção de Perigos:** O sistema é projetado de tal forma que o perigo é fisicamente ou logicamente impossível. *Exemplo: Uma cortadora de papel industrial que só funciona se os dois botões de acionamento forem pressionados simultaneamente pela mesma pessoa, garantindo que as mãos dela não estejam na lâmina.*
2. **Deteção e Remoção de Perigos:** O sistema detecta que um estado perigoso ocorreu e atua para interrompê-lo antes que se torne um acidente. *Exemplo: Uma válvula de alívio em um tanque químico que abre automaticamente se a pressão exceder o limite seguro.*
3. **Limitação de Danos:** Assume-se que o acidente ocorrerá, e o foco é minimizar o prejuízo. *Exemplo: Sistemas de extintores automáticos no compartimento do motor de um avião ou a existência de coletes salva-vidas.*

8.13 Cultura de Segurança e “Acidentes Normais”

Para concluir este capítulo, é imperativo mencionar a teoria de **Charles Perrow (1984)** sobre os chamados “Acidentes Normais”. Segundo Perrow, em sistemas de alta complexidade e com acoplamento forte (onde uma falha em uma parte se propaga instantaneamente para outra), os acidentes não são anomalias, mas sim propriedades estatisticamente inevitáveis do próprio sistema.

Sistemas complexos possuem interações não previstas. Uma falha trivial (ex: falha em um sensor de temperatura) pode causar um comportamento inesperado em outro componente (ex: o software reduz a rotação de um motor para esfriar), o que por sua vez gera um novo sinal de erro que confunde o operador humano.

A lição da engenharia moderna de segurança não é tentar eliminar 100% dos riscos — o que é impossível — mas sim criar sistemas **resilientes**. Isso significa que o software

deve ser robusto o suficiente para dar visibilidade clara ao operador sobre o estado real do sistema e evitar que falhas em cascata se tornem incontroláveis.

8.14 O Registro de Perigos (*Hazard Registry*)

Todo o trabalho de análise e mitigação deve ser formalizado no **Registro de Perigos**. Este documento é a prova legal e técnica de que a equipe de engenharia exerceu o seu “dever de cuidado”. Em caso de acidentes, o registro demonstra que os perigos foram previstos e que medidas de mitigação foram projetadas de acordo com as normas vigentes. Sem um registro de perigos atualizado, o software não pode ser certificado para uso em domínios críticos como aeroespacial, nuclear ou médico.

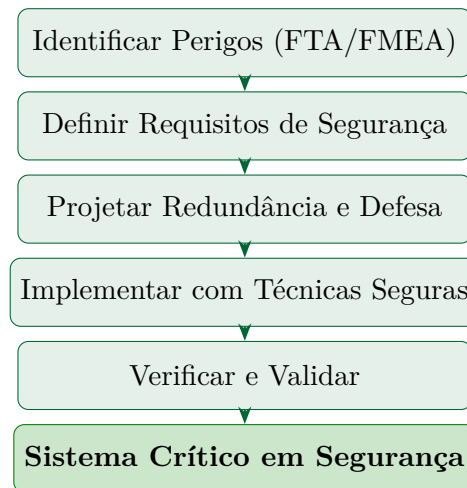


Figura 8.9: Resumo do Ciclo de Vida do Desenvolvimento de Software Crítico em Segurança.

Referências Bibliográficas

BEYER, Betsy; JONES, Chris et al. **Site Reliability Engineering: How Google Runs Production Systems**. Sebastopol, CA: O'Reilly Media, 2016. ISBN 978-1-491-92912-4. Disponível em: <<https://sre.book.google.com/>>.

BEYER, Betsy; MURPHY, Niall Richard et al. **The Site Reliability Workbook: Practical Ways to Implement SRE**. Sebastopol, CA: O'Reilly Media, 2018. ISBN 978-1-492-02953-4.

FISHER, Nate. **To the Moon and Back: The Margaret Hamilton Story**. Cambridge, MA: MIT Press, 2019.

GARMAN, John R. The “Bug” Heard 'Round the World. **ACM SIGSOFT Software Engineering Notes**, v. 6, n. 5, p. 3–10, 1981.

GOOGLE SRE TEAM. **Google Site Reliability Engineering**. 2024. Disponível em: <<https://sre.google/>>. Acesso em: 1 mar. 2024.

HAMILTON, Margaret H.; ZELDIN, Saydean. **Higher Order Software – A Methodology for Defining Software**. [S.l.]: IEEE, 1976. SE-2, p. 9–32.

HUMBLE, Jez; FARLEY, David. **Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation**. Upper Saddle River, NJ: Addison-Wesley, 2010. ISBN 978-0-321-60191-9.

KIM, Gene et al. **The DevOps Handbook: How to Create World-Class Agility, Reliability, and Security in Technology Organizations**. 2. ed. Portland, OR: IT Revolution Press, 2018. ISBN 978-1-942-78851-0.

MCKINNEY, Wes. **Python para Análise de Dados: Tratamento de Dados com Pandas, NumPy e Jupyter**. 3. ed. São Paulo: O'Reilly Brasil, 2022. ISBN 978-8-550-81928-0.

SOMMERVILLE, Ian. **Engenharia de Software**. 10. ed. São Paulo: Pearson, 2019. ISBN 978-8-543-02469-7.